

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий _____

Кафедра Программной инженерии _____

Специальность 6-05-0612-01 Программная инженерия (профилизация Программное
обеспечение информационных технологий) _____

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ НА ТЕМУ:**

**«Web-приложение для проектирования архитектуры на основе бизнес-
требований»**

Выполнил студент _____ Филиппук Илья Андреевич
(Ф.И.О.)

Руководитель работы _____ к.т.н., доц. Смелов В.В.
(учен. степень, звание, должность, Ф.И.О., подпись)

Зав. кафедрой _____ к.т.н., доц. Смелов В.В.
(учен. степень, звание, должность, Ф.И.О., подпись)

Курсовая работа защищена с оценкой _____

Оглавление

Введение.....	4
1 Постановка задачи и обзор аналогичных решений.....	5
1.1 Постановка задачи.....	5
1.2 Обзор аналогов.....	6
1.2.1 Анализ веб-приложения «AWS Well-Architected Tool».....	6
1.1.2 Аналог Cloudcraft.....	8
1.3 Вывод по разделу.....	11
2 Проектирование веб-приложения.....	12
2.1 Функциональные возможности.....	12
2.2 Логическая схема базы данных.....	13
2.3 Выбор платформы и технологий.....	19
2.4 Архитектура приложения.....	20
2.5 Выводы по разделу.....	21
3 Реализация веб-приложения.....	23
3.1 Программная платформа.....	23
3.2 Реляционная база данных PostgreSQL.....	23
3.3 Программные библиотеки.....	24
3.4 Структура серверной части.....	25
3.5 Реализация функций.....	26
3.6 Структура клиентской части.....	32
3.7 Настройка конфигурации Docker.....	33
3.8 Настройка конфигурации Nginx.....	34
3.9 Выводы по разделу.....	35
4 Тестирование веб-приложения.....	37
4.1 Функциональное тестирование.....	37
4.2 Автоматизированное тестирование.....	38
4.3 Нагрузочное тестирование.....	39
4.4 Выводы по разделу.....	40
5. Руководство пользователя.....	41
5.1 Авторизация.....	41
5.2. Прохождение опроса о продукте.....	42
5.3 Генерация архитектуры.....	43
5.4 Личный кабинет.....	44
5.5 Панель администратора.....	45

5.6 Развёртывание и проверка работоспособности	47
5.7 Выводы по разделу	48
Заключение.....	49
Список использованной литературы	50
Приложение А.....	51
Приложение Б	52
Приложение В	53
Приложение Г	54
Приложение Д.....	55
Приложение Е	56
Приложение Ж	58

Введение

В условиях активного развития цифровых продуктов стартапы и небольшие компании всё чаще сталкиваются с необходимостью быстрого выбора подходящей архитектуры программного обеспечения и модели развёртывания будущей системы. На ранних этапах проектирования разработчики и заказчики нередко принимают решения на основе ограниченного опыта или без достаточного учёта предполагаемой нагрузки, бюджета, состава команды и функциональных требований. Это может приводить к избыточной сложности системы, нерациональному использованию инфраструктурных ресурсов и дополнительным финансовым затратам. В связи с этим возникает необходимость в разработке специализированного программного средства, которое позволит на основе исходных параметров проекта формировать обоснованные рекомендации по архитектуре приложения, составу технологического стека и этапам дальнейшего развития системы.

Целью работы является создание веб-приложения для проектирования архитектуры на основе бизнес-требований, предназначенного для автоматизации процесса выбора архитектуры программного обеспечения, модели развёртывания и базовых инфраструктурных решений на ранних этапах проектирования информационной системы.

Для достижения цели необходимо выполнить следующие задачи:

- изучить предметную область проектирования программной архитектуры и проанализировать существующие подходы и аналоги систем поддержки архитектурного планирования (глава 1);

- спроектировать архитектуру веб-приложения, включая структуру базы данных и логику REST (глава 2);

- программно реализовать клиентскую и серверную части приложения на выбранном стеке технологий (глава 3);

- протестировать функционал системы и убедиться в корректности её работы (глава 4);

- подготовить руководство пользователя с описанием основных сценариев работы (глава 5).

- подготовить руководство администратора с описанием развёртывания системы (глава 6).

Приложение предназначено для двух категорий пользователей: администраторов, осуществляющих контроль за работой системы и мониторингом действий пользователей, а также клиентов, которым необходимо получить готовое архитектурное решение для их проекта.

Для реализации проекта выбраны следующие технологии: серверная часть построена на платформе Node.js, клиентская часть реализована с использованием библиотеки React [1] в формате одностраничного приложения, для хранения данных используется реляционная СУБД PostgreSQL [2], в качестве прокси-сервера применяется Nginx [3], а развёртывание осуществляется с помощью Docker Compose [4].

1 Постановка задачи и обзор аналогичных решений

1.1 Постановка задачи

На ранних этапах разработки программных продуктов одной из наиболее значимых задач является выбор рациональной архитектуры системы, модели её развёртывания и базового набора технологических компонентов. На практике такие решения нередко принимаются на основе субъективного опыта разработчика или общих представлений о проекте, без достаточного учёта предполагаемой нагрузки, функциональных требований, состава команды, ограничений по бюджету и особенностей предметной области. В результате выбранные архитектурные решения могут оказаться избыточными, экономически неэффективными либо недостаточно масштабируемыми для дальнейшего развития системы.

Дополнительная сложность заключается в том, что начальное архитектурное планирование требует сопоставления бизнес-требований с техническими параметрами будущего программного продукта. При отсутствии специализированного инструмента данный процесс выполняется вручную, что увеличивает вероятность ошибок, затрудняет сравнение альтернативных вариантов и снижает обоснованность принимаемых решений. Особенно актуальна данная проблема для стартапов и небольших команд, которые ограничены во времени и ресурсах, но при этом нуждаются в быстром получении понятных и аргументированных рекомендаций по построению программной архитектуры.

Для решения этих проблем разрабатывается веб-приложение для проектирования архитектуры на основе бизнес-требований ArchitecturePlanner. Приложение предназначено для автоматизации процесса первичного архитектурного планирования программных систем, обеспечивая централизованный сбор исходных параметров проекта, их обработку с использованием детерминированного набора правил и формирование рекомендаций по выбору архитектуры приложения, модели развёртывания, состава технологических компонентов, ориентировочной стоимости инфраструктуры и этапов дальнейшего развития системы.

Важным требованием к разрабатываемой системе является обоснованность и воспроизводимость получаемых результатов. Это означает, что при одинаковых входных параметрах приложение должно формировать одинаковые рекомендации.

В системе предусматриваются две основные роли пользователей. Клиент заполняет анкету проекта, получает сформированный архитектурный план, сохраняет результаты и просматривает ранее созданные варианты. Администратор, в свою очередь, управляет пользователями системы, просматривает сводную аналитическую информацию и настраивает параметры движка рекомендаций, влияющие на формирование итоговых результатов.

Система должна обеспечивать хранение сформированных архитектурных планов и связанных с ними исходных данных, что позволяет возвращаться к ранее созданным решениям. Дополнительно предусматривается возможность представления результата в наглядной форме, включая краткое текстовое описание,

дорожную карту реализации, оценку затрат и экспортируемую диаграмму архитектуры.

1.2 Обзор аналогов

В данном разделе проводится анализ существующих систем и инструментов, предназначенных для архитектурного планирования программных решений, выбора технологического стека и оценки инфраструктурных параметров проекта. Цель анализа заключается в изучении их функциональных возможностей, подходов к формированию рекомендаций, а также в определении требований к разрабатываемому веб-приложению.

В ходе анализа будут выявлены основные функциональные возможности рассматриваемых решений, их достоинства и ограничения с точки зрения применения на ранних этапах проектирования программных приложений. На основе полученных результатов будут сформулированы требования к разрабатываемому приложению с учётом выявленных преимуществ существующих подходов и необходимости устранения их недостатков.

1.2.1 Анализ веб-приложения «AWS Well-Architected Tool»

AWS Well-Architected Tool представляет собой облачный сервис компании Amazon Web Services, предназначенный для оценки архитектуры программных систем на основе набора рекомендуемых практик AWS [5]. Система используется для документирования архитектурных решений, выявления потенциальных рисков и формирования рекомендаций по повышению надёжности, безопасности, производительности и экономической эффективности разрабатываемого решения [6]. Доступ к сервису осуществляется через консоль управления AWS Management Console.

Главная страница системы содержит панель мониторинга, предоставляющую сводную информацию по анализируемым рабочим нагрузкам и выявленным архитектурным рискам, что представлено на рисунке 1.1.

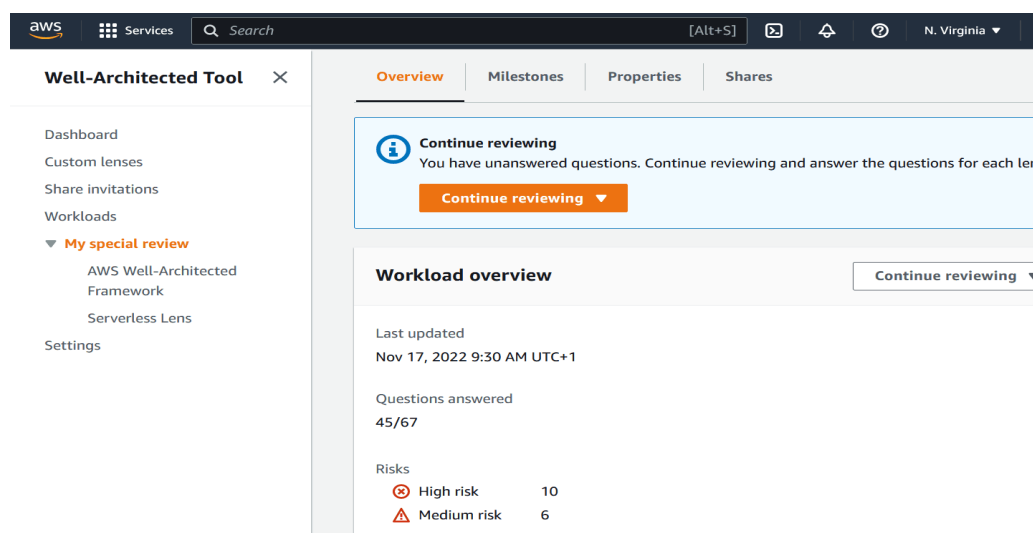


Рисунок 1.1 – главная страница AWS Well-Architected Tool

На панели отображаются общее количество рабочих нагрузок, число проблем высокого и среднего уровня риска, распределение замечаний по архитектурным направлениям, а также сведения о состоянии мероприятий по улучшению системы.

Такой подход позволяет использовать AWS Well-Architected Tool как средство контроля качества архитектурных решений и поддержки их поэтапного совершенствования.

Для анализа различных архитектурных решений, пользователь создаёт отдельные рабочие нагрузки, то есть описания конкретных систем или проектов, и для каждой из них ведётся отдельный архитектурный анализ, что представлено на рисунке 1.2.

	Name	Owner	Overall status	High risks	Medium risks	Improvement status	Last updated
☐	Bootcamp - Febrero 23 24 25 - 2021		18/67	16	2	None	July 25, 2021 8:26 PM (UTC+2)
☐	Bootcamp - September 28,29,1 - 2020		52/58	25	12	Not Started	November 10, 2021 6:48 PM (UTC+2)
☐			58/58	13	15	None	October 7, 2022 3:56 PM (UTC+2)
☐			58/58	15	12	None	October 22, 2022 12:21 AM (UTC+2)
☐	MCO-MEX-WAPP-Bootcamp		61/67	46	14	None	February 23, 2022 6:55 PM (UTC+2)
☐	Octank data ingestion workload		9/61	8	1	None	November 10, 2021 6:37 PM (UTC+2)
☐	Octank pre-Prod review		1/67	1	0	None	February 23, 2022 6:50 PM (UTC+2)
☐	WAPP Bootcamp - 20200727		53/67	39	12	None	February 24, 2021 6:35 PM (UTC+2)
☐	WorkLoad Test		1/67	0	0	None	October 5, 2022 7:36 PM (UTC+2)
☐			58/58	52	6	None	August 10, 2022 11:43 PM (UTC+2)

Рисунок 1.2 – Страница рабочих нагрузок

При работе с данным инструментом пользователь может воспользоваться механизмом архитектурного обзора с помощью «линз». Система проверяет архитектуру не в общем виде, а по наборам критериев и вопросов. Для одной нагрузки можно применять несколько линз, в том числе пользовательские. Это важная особенность, потому что она показывает структурированный подход к оценке архитектуры, что представлено на рисунке 1.3.

Well Architected Tool

Dashboard Custom Lenses Custom Lens

Lens Name FileMaker Base Assessment

Description For performing a high level assessment of FileMaker workloads and deployments.

You can define up to 10 pillars in a custom lens.

Pillars	# of Questions
1 Operational Excellence	2
2 Security	3
3 Reliability	3
4 Performance Efficiency	3
5 Cost Optimization	2
6 Sustainability	1

Add Default Pillars Add New Pillar

Custom Lens Export Create JSON File

Рисунок 1.3 – Архитектурный обзор по «линзам»

После анализа система формирует список рекомендаций по улучшению архитектуры. Данная функциональная особенность также позволяет получить список потенциальных точек отказа, а также рекомендации по их исправлению, что представлено на рисунке 1.4.

Improvement item	Pillar	Risk	Applicable workloads
☐ Accommodate fixed service quotas and constraints through architecture	Reliability	High	2
☐ Analytics	Reliability	High	3
☒ Automate quota management	Reliability	High	2
☐ Automate responses (Real-time processing and alarming)	Reliability	High	2
☐ Aware of service quotas and constraints	Reliability	High	2
☐ Build services focused on specific business domains and functionality	Reliability	High	3
☐ Choose how to segment your workload	Reliability	High	2
☐ Conduct game days regularly	Reliability	High	1
☐ Conduct reviews regularly	Reliability	High	3
☐ Control and limit retry calls	Reliability	High	3

Automate quota management	
Pillar	Reliability
Applicable workloads (2)	Example Company - Serverless - Pre-Production Example Company - Serverless - Production

Рисунок 1.4 – Рекомендации по улучшению архитектуры

К преимуществам AWS Well-Architected Tool можно отнести структурированный подход к анализу архитектуры, наличие рекомендаций по улучшению системы, возможность сохранения контрольных вех и поддержку совместной работы над проектом. Инструмент позволяет выявлять архитектурные риски и документировать результаты анализа, что делает процесс проектирования более формализованным и обоснованным.

К недостаткам данного инструмента следует отнести его жёсткую ориентированность на экосистему AWS, из-за чего его применение для проектов, не связанных с инфраструктурой Amazon, является ограниченным. Кроме того, система в большей степени предназначена для оценки уже спроектированных или развёрнутых решений, чем для автоматического подбора архитектуры на самых ранних этапах проектирования.

1.1.2 Аналог Cloudcraft

Cloudcraft представляет собой веб-платформу для проектирования, визуализации и документирования облачной архитектуры, ориентированную преимущественно на инфраструктуры AWS и Azure [7]. Система предназначена для создания архитектурных диаграмм, оценки предполагаемых затрат на облачную инфраструктуру, а также совместной работы над архитектурными решениями. Cloudcraft распространяется как облачный сервис и используется как для ручного проектирования архитектуры, так и для автоматического построения диаграмм на основе уже существующей облачной инфраструктуры.

Главная страница системы предоставляет пользователю доступ к средствам визуального моделирования архитектуры, функциям автоматического построения диаграмм, инструментам оценки затрат и возможностям совместной работы, что представлено на рисунке 1.5.

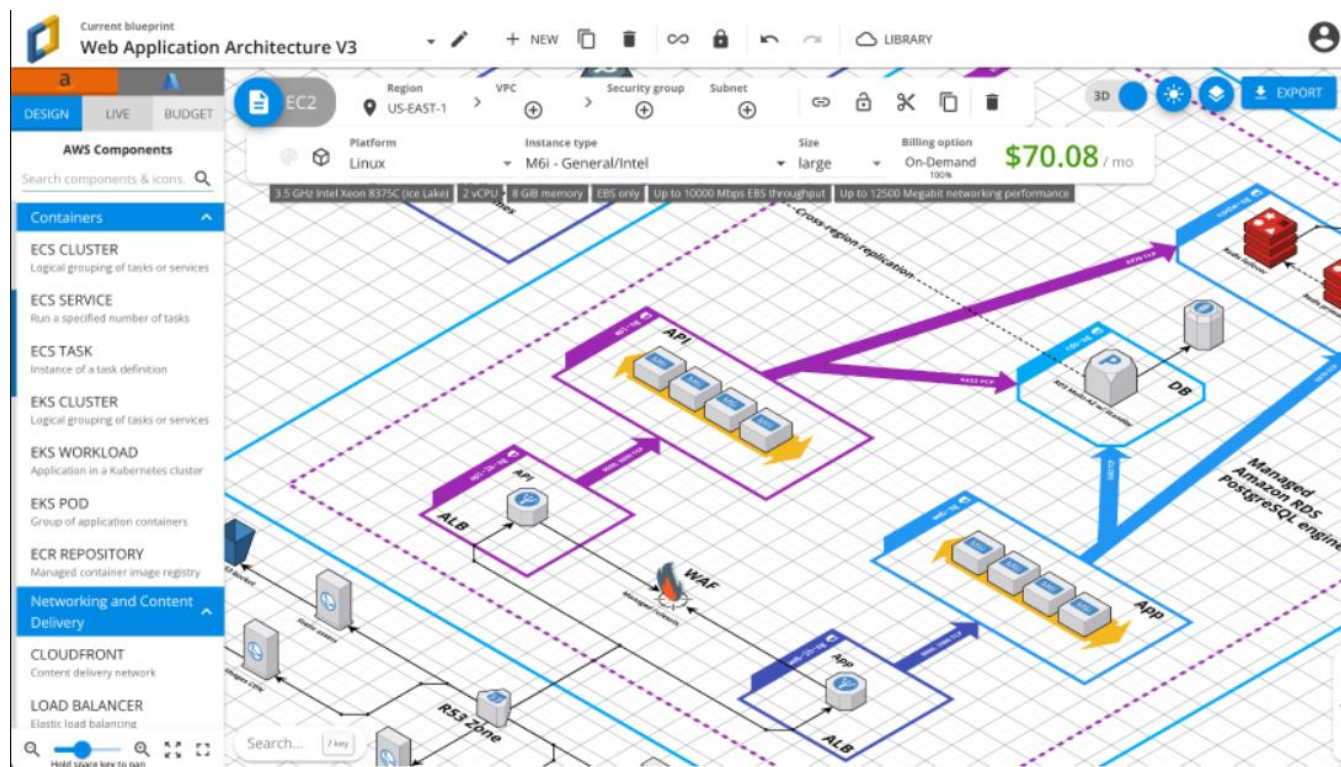


Рисунок 1.5 – Главная страница Cloudcraft

Основное внимание в Cloudcraft уделяется графическому представлению инфраструктуры (рисунок 1.6), её документированию и анализу изменений, что делает систему удобной для подготовки архитектурных схем, технической документации и предварительной оценки облачных расходов.

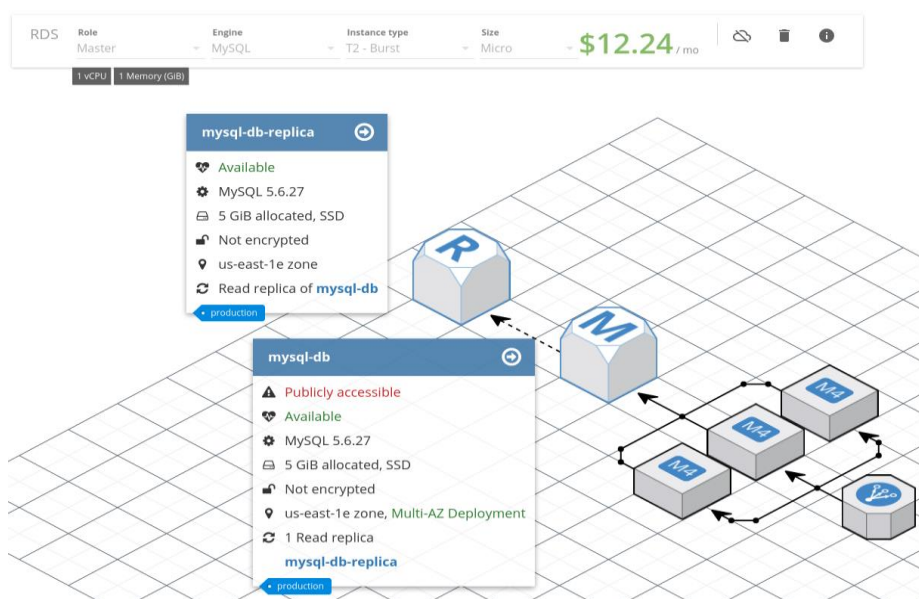


Рисунок 1.6 – Графическое представление инфраструктуры

Приложение также предоставляет возможность автоматического построения диаграмм существующей облачной инфраструктуры с помощью Live scan. Данная функция позволяет воспользоваться наиболее подходящими в конкретной ситуации архитектурными решениями, сгенерированными системой автоматически на основе данных о проекте, что представлено на рисунке 1.7.

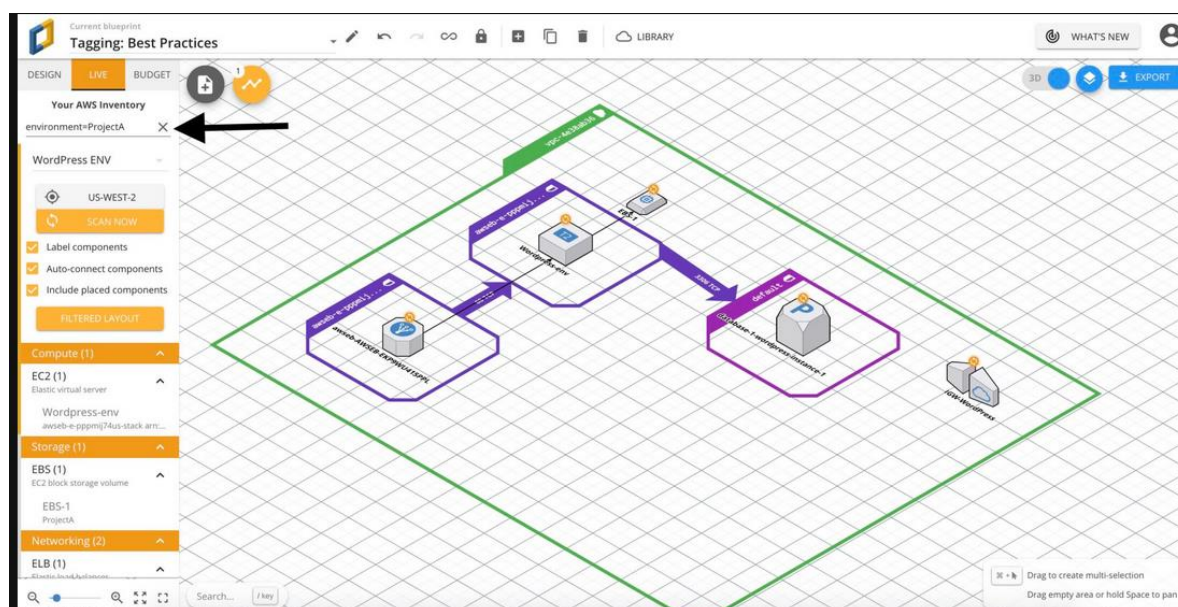


Рисунок 1.7 – Автоматическое построение диаграмм с помощью Live scan

В процессе построения диаграмм архитектуры приложением также выполняется автоматическая оценка стоимости развёртывания архитектуры и анализ изменений затрат при модификации схемы, что представлено на рисунке 1.8. Данная функциональная особенность позволяет удобно подстраивать выбираемые архитектурные решения под доступный бюджет проекта.

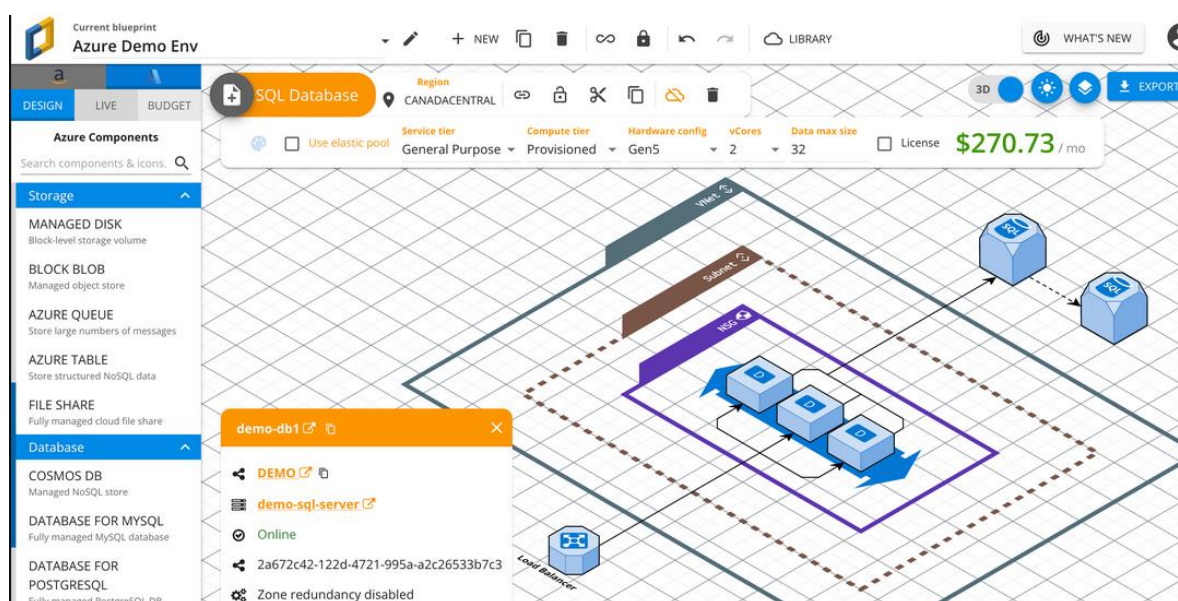


Рисунок 1.8 – Автоматическая оценка стоимости развёртывания

К преимуществам Cloudcraft можно отнести наглядное визуальное представление облачной архитектуры, наличие автоматического построения диаграмм, поддержку оценки инфраструктурных затрат, а также средства совместной работы и документирования. Система удобна для подготовки профессиональных архитектурных схем и позволяет поддерживать их актуальность при изменении облачной инфраструктуры.

К недостаткам Cloudcraft следует отнести его узкую ориентированность на облачные платформы AWS и Azure, что ограничивает применение системы для более широкого круга программных проектов. Кроме того, Cloudcraft в первую очередь предназначен для визуализации и документирования уже выбранной или существующей инфраструктуры, а не для автоматического формирования архитектурных рекомендаций на основе исходных параметров проекта.

1.3 Вывод по разделу

1. Анализ существующих веб-приложений для построения архитектуры программных продуктов позволил выделить ряд недостатков: узкая ориентированность на облачные платформы AWS и Azure [8], и заточенность на визуализацию и документирование уже выбранной или существующей инфраструктуры.

2. На основе выделенных недостатков и данных листа задания разрабатываемое веб-приложение должно поддерживать функциональность, позволяющую определять оптимальные архитектурные решения для ещё не сформированных или не задокументированных инфраструктур.

3. В приложении необходимо реализовать полноценную систему генерации инфраструктурных диаграмм с возможностью сохранять полученные диаграммы в удобных для этого форматах.

2 Проектирование веб-приложения

2.1 Функциональные возможности

Для описания функциональных требований к системе была построена диаграмма вариантов использования, которая размещена в Приложении А. Диаграмма отражает взаимодействие двух категорий пользователей с системой: администратора и клиента.

Более подробное описание ролей представлено в таблице 2.1.

Таблица 2.1 – Описание ролей приложения

Роль	Описание
Администратор	Управление пользователями (редактирование, удаление, изменение данных), управление настройками движка (корректировка коэффициентов, отключение и включение модулей, управление каталогом технологий), получение отчётности о работе системы и действиях пользователей
Клиент	Прохождение опроса о проекте, для которого будет генерироваться архитектурная диаграмма, получение сгенерированной архитектуры и рекомендованного стека технологий с обоснованием выбора, получение плана реализации проекта, а также оценочной стоимости развёртывания, возможность экспорта диаграмм в SVG или DRAWIO

На основе выделенных ролей были выявлены основные функции системы, которые описаны в таблице 2.2.

Таблица 2.2 – Основные функции системы

№	Функция	Роль	Описание
1	Авторизация	Клиент	Вход в систему с использованием учётных данных
2	Прохождение опроса о проекте	Клиент	Заполнение данных о проекте, для которого будет генерироваться архитектурная диаграмма, план реализации и оценочная стоимость
3	Получение сгенерированной архитектуры	Клиент	Получение плана реализации, оценочной стоимости, стека технологий и сгенерированной диаграммы
4	Экспорт архитектуры	Клиент	Экспорт архитектуры в SVG и DRAWIO
5	Управление пользователями	Администратор	Управление данными пользователя, управление учётными записями пользователей, управление архитектурами пользователей
6	Управление каталогом технологий	Администратор	Включение и отключение определённых модулей движка построения архитектуры
7	Управление и редактирование алгоритма генерации	Администратор	Настройка движка построения архитектуры, изменение коэффициентов

Продолжение таблицы 2.2 – Основные функции системы

№	Функция	Роль	Описание
8	Получение аналитики системы	Администратор	Получение отчёта о работе системы и активности пользователей

2.2 Логическая схема базы данных

Логическая схема базы данных состоит из 11 таблиц, каждая из которых отвечает за хранение определённых данных. Полная логическая схема базы данных представлена в . Наименование и описание всех сущностей базы данных представлено в таблице 2.3.

Таблица 2.3 – Описание сущностей базы данных

Название таблицы	Назначение
Users	Хранит список пользователей
Admin_audit_log	Хранит изменения, внесённые администратором
Engine_settings	Хранит настройки движка рекомендаций
Generated_plans	Хранит все планы, сгенерированные всеми пользователями
Region_data_cache	Хранит специфические особенности развёртывания в конкретных регионах
Plan_runs	Хранит все планы, принадлежащие пользователю
Plan_run_technologies	Хранит список технологий, задействованных в конкретном проекте
Plan_components	Хранит модули, включённые в генерацию плана
Projects	Хранит все проекты, доступные пользователю
Technologies	Хранит список технологий
Technology_categories	Вспомогательная таблица для соединения технологий и категорий

Далее подробно описаны все основные сущности базы данных.

Описание структуры таблицы Users, которая содержит информацию о пользователях системы, приведено в таблице 2.4.

Таблица 2.4 – Структура таблицы Users

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный идентификатор пользователя (первичный ключ)
Auth0_sub	text	Идентификатор, полученный пользователем при регистрации
Email	text	Адрес электронной почты пользователя
Display_name	text	Имя профиля пользователя в системе
Role	text	Роль пользователя в системе
Created_at	timestamp	Время создания профиля пользователя

Таблица Admin_audit_log содержит информацию об изменениях, внесённых администратором в систему или любых иных действиях, произведённых администратором. Более подробное описание структуры данной таблицы приведено в таблице 2.5.

Таблица 2.5 – Структура таблицы Admin_audit_log

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор записи (первичный ключ)
Actor_user_id	Bigint	Уникальный числовой идентификатор администратора, внесшего изменение (вторичный ключ)
Action	Text	Внесённое администратором изменение
Target_type	Text	Объект, к которому применяется изменение
Details_json	Jsonb	Детали изменения в формате json
Created_at	timestampz	Время внесения изменения

Таблица Engine_settings содержит информацию о текущих настройках движка рекомендаций. Описание структуры данной таблицы представлено в таблице 2.6.

Таблица 2.6 – Структуры таблицы Engine_settings

Поле	Тип данных	Назначение
Key	Text	Наименование настройки движка (первичный ключ)
Value_json	Jsonb	Значение настройки, соответствующее наименованию
Updated_by	Bigint	Отметка о последнем администраторе, изменившем настройку (вторичный ключ)
Updated_at	timestampz	Время последнего изменения настройки

Таблица Generated_plans содержит информацию о архитектурных планах, сгенерированных пользователями системы. Описание структуры данной таблицы представлено в таблице 2.7.

Таблица 2.7 – Структура таблицы Generated_plans

Поле	Тип данных	Назначение
Id	Bingserial	Уникальный числовой идентификатор сгенерированного плана (первичный ключ)
User_id	Bigint	Уникальный числовой идентификатор пользователя, сгенерировавшего план (вторичный ключ)
Plan_id	Text	Идентификатор плана реализации архитектуры
Project_name	Text	Наименование проекта
Input_payload_json	Jsonb	Данные, введённые пользователем о проекте
Summary	Text	Обобщённая информация о проекте

Продолжение таблицы 2.7 – Структура таблицы Generated_plans

Поле	Тип данных	Назначение
Recommendation_payload	Jsonb	Рекомендации по архитектурным решениям
Region_profile_payload	Jsonb	Специфика развёртывания приложения в зависимости от региона
Roadmap_payload	Jsonb	План развёртывания приложения
Development_plan_payload	Jsonb	План разработки архитектуры приложения
Cost_payload	Jsonb	Оценочная стоимость развёртывания приложения
Diagram_payload	Jsonb	Данные для построения архитектурной диаграммы приложения
Drawio_xml	Text	Данные для построения диаграммы в формате DRAWIO
Created_at	Timestamptz	Время создания плана

Таблица Region_data_cache содержит информацию о специфике развёртывания приложения в зависимости от региона. Описание структуры данной таблицы представлено в таблице 2.8.

Таблица 2.8 – Структура таблицы Region_data_cache

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор записи
Region_code	Text	Код региона, для которого описывается специфика
Source_name	Text	Имя источника получения информации
Payload	Jsonb	Информация о специфике развёртывания для региона
Refreshed_at	Timestamptz	Время последнего обновления записи

Таблица Plan_runs содержит информацию об архитектурных планах, создаваемых пользователем. Описание структуры данной таблицы представлено в таблице 2.9.

Таблица 2.9 – Структура таблицы Plan_runs

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор плана (первичный ключ)
Project_id	Bigint	Уникальный числовой идентификатор соответствующего проекта

Продолжение таблицы 2.9 – Структура таблицы Plan_runs

Поле	Тип данных	Назначение
User_id	Bigint	Уникальный числовой идентификатор пользователя (вторичный ключ)
Plan_id	Text	Идентификатор генерируемого плана
Project_name_snapshot	Text	Имя создаваемого проекта
Input_payload	Jsonb	Данные, введённые пользователем при опросе
Summary	Text	Обобщённая информация о проекте
Recommendation_payload	Jsonb	Значение рекомендаций по проекту
Region_profile	Jsonb	Значение региональной специфики
Roadmap_payload	Jsonb	Информация по дорожной карте развития проекта
Development_plan_payload	Jsonb	Информация по плану разработки проекта
Cost_payload	Jsonb	Информация по денежной оценке компонентов проекта
Diagram_payload	Jsonb	Данные для построения диаграммы в формате SVG
Drawio_xml	Text	Данные для построения диаграммы в формате DRAWIO
Architecture_style	Text	Информация об архитектурном стиле проекта
Deployment_model	Text	Информация о модели развёртывания проекта
Target_region	Text	Целевой регион развёртывания проекта
Monthly_estimate	Integer	Помесячная оценка стоимости поддержания проекта
Created_at	Timestamptz	Отметка о времени создания записи

Таблица Plan_run_technologies содержит информацию о технологиях, используемых в проекте с обоснованием использования. Описание структуры данной таблицы представлено в таблице 2.10.

Таблица 2.10 – Структура таблицы Plan_run_technologies

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор технологии для проекта (первичный ключ)
Plan_run_id	Bigint	Уникальный числовой идентификатор соответствующего плана (вторичный ключ)

Продолжение таблицы 2.10 – Структура таблицы Plan_run_technologies

Поле	Тип данных	Назначение
Technology_id	Bigint	Уникальный числовой идентификатор технологии из списка (вторичный ключ)
Justification	Text	Обоснование применения технологии
Created_at	Timestamptz	Отметка о времени создания записи

Таблица Plan_components содержит информацию о компонентах, включаемых пользователем в процессе генерации архитектуры. Структура данной таблицы представлена в таблице 2.11.

Таблица 2.11 – Структура таблицы Plan_components

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор компонента (первичный ключ)
Plan_run_id	Bigint	Уникальный числовой идентификатор соответствующего плана (вторичный ключ)
Component_code	Text	Код включаемого компонента
Created_at	Timestamptz	Отметка о времени создания записи

Таблица Projects содержит в себе полную информацию о создаваемых пользователями проектах. Структура данной таблицы представлена в таблице 2.12.

Таблица 2.12 – Структура таблицы Projects

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор проекта (первичный ключ)
User_id	Bigint	Уникальный числовой идентификатор пользователя (вторичный ключ)
Project_name	Text	Наименование проекта
Created_at	Timestamptz	Отметка о времени создания записи в таблице
Updated_at	Timestamptz	Отметка о времени последнего обновления записи в таблице

Таблица Technologies содержит информацию о каталоге технологий, используемых в системе и добавляемых администраторами. Структура данной таблицы представлена в таблице 2.13.

Таблица 2.13 – Структура таблицы Technologies

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор технологии
Name	Text	Наименование технологии
Category	Text	Наименование категории, которой принадлежит технология
Description	Text	Описание технологии
Logo_url	Text	Ссылка на изображение, соответствующее технологии
Is_active	Boolean	Статус технологии в процессе генерации
Created_at	Timestamptz	Отметка о времени создания записи в таблице
Updated_at	Timestamptz	Отметка о времени последнего обновления записи в таблице

Таблица Technology_categories является вспомогательной таблицей для категорий и технологий, соответствующих этим категориям. Структура данной таблицы представлена в таблице 2.14.

Таблица 2.14 – Структура таблицы Technology_categories

Поле	Тип данных	Назначение
Id	Bigserial	Уникальный числовой идентификатор
Code	Text	Код категории
Name	Text	Наименование категории
Sort_order	Integer	Порядок сортировки категорий
Is_active	Boolean	Отметка об активности категории
Created_at	Timestamptz	Отметка о времени создания записи
Updated_at	Timestamptz	Отметка о времени последнего обновления записи

Связи между вышеописанными таблицами представлены в таблице 2.14.

Таблица 2.15 – Связи между таблицами базы данных

Таблица-источник	Связанная таблица	Тип связи
Users	Projects	Один ко многим
Users	Engine_settings	Один ко многим
Users	Plan_runs	Один ко многим
Plan_runs	Plan_components	Один ко многим
Plan_runs	Plan_run_technologies	Один ко многим
Technologies	Technology_categories	Один ко многим
Plan_run_technologies	Technologies	Один ко многим

Разработанная схема базы данных включает в себя 11 таблиц, что полностью покрывает все функциональные требования к системе.

2.3 Выбор платформы и технологий

Разрабатываемое веб-приложение предназначено для работы в современных браузерах, что обеспечивает кроссплатформенность и доступность на различных устройствах.

Бэкенд (серверная часть):

- Node.js (версия 20 LTS) – платформа для выполнения JavaScript на сервере, обеспечивающая асинхронную обработку запросов.

- Express (версия 4.21.0) – веб-фреймворк для Node.js, упрощающий создание REST API.

- Drizzle (версия 1.0.0. beta) – построитель SQL-запросов и инструмент миграций для работы с реляционной базой данных.

- express-oauth2-jwt-bearer (версия 9.0.2) – реализация JWT для аутентификации.

- express-validator (версия 7.2.0) – валидация входных данных запросов к API.

- pg (версия 8.13.0) – драйвер для подключения к PostgreSQL.

Фронтенд (клиентская часть):

- React (версия 19.2.4) – библиотека для построения пользовательских интерфейсов.

- React Router DOM (версия 7.13.1) – маршрутизация в клиентском приложении.

- Axios (версия 1.13.6) – HTTP-клиент для запросов к серверу.

- Vite (версия 8.0.0) – сборка и dev-сервер клиентского приложения.

- i18next и react-i18next (версии 25.8.18 и 16.5.8) – многоязычный интерфейс приложения.

База данных:

- PostgreSQL 16 (образ Alpine в Docker Compose) – реляционная СУБД, обеспечивающая ACID-транзакции и надёжное хранение данных.

Инфраструктура:

- Docker и Docker Compose – контейнеризация приложения (сервисы API, база данных, Nginx).

- Nginx (официальный образ nginx:alpine) – веб-сервер для раздачи статических файлов фронтенда и обратного проксирования к API.

Таким образом, выбранный стек технологий обеспечивает масштабируемость, безопасность данных и удобство развёртывания приложения с помощью контейнеризации.

2.4 Архитектура приложения

Архитектура веб-приложения построена по клиент-серверному принципу с использованием REST API. Архитектурная схема представлена на рисунке 2.1 и в Приложении В.

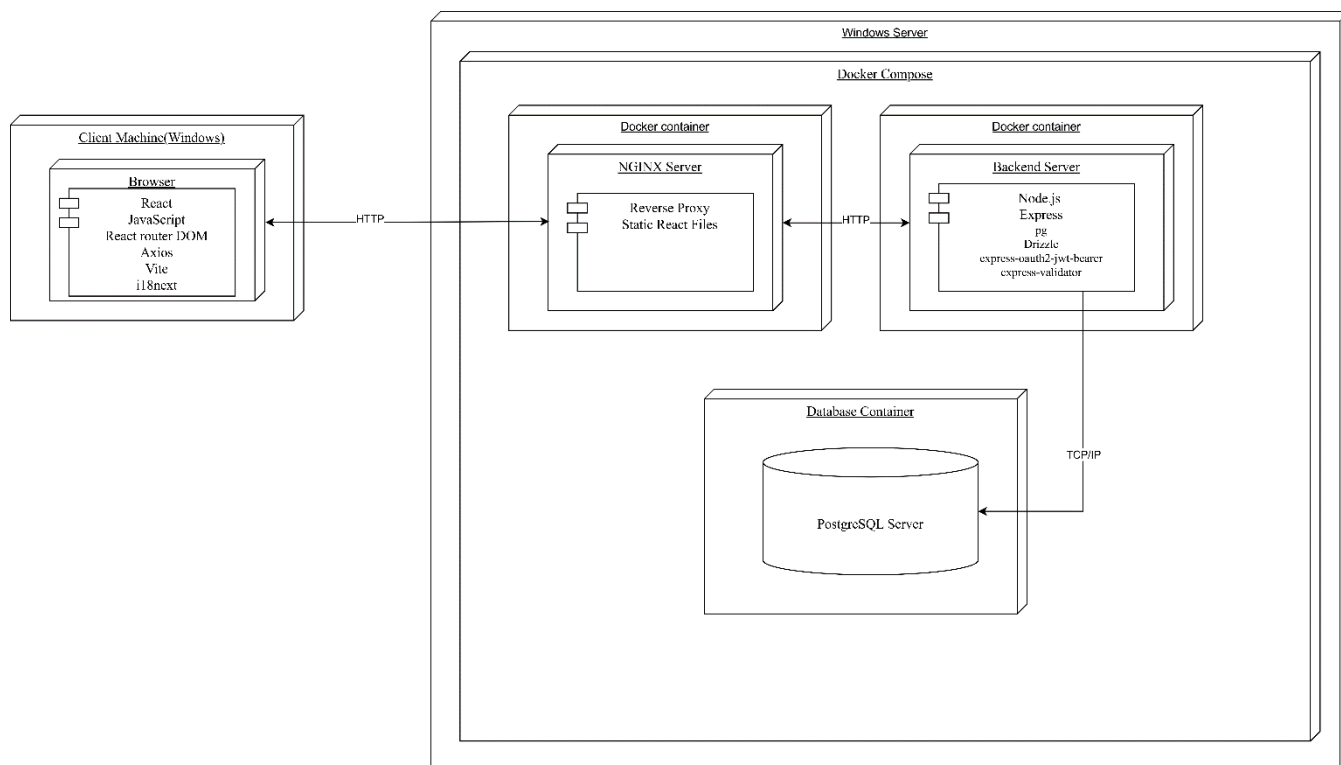


Рисунок 2.1 – Архитектура приложения

В таблице 2.16 подробно описаны ключевые компоненты архитектуры разработанного веб-приложения, их технологическая основа и функциональная роль в системе.

Таблица 2.16 – Основные компоненты архитектуры

Компонент	Технология	Назначение
Клиент	React (Vite), React Router DOM, Axios	Формирование пользовательского интерфейса, маршрутизация между страницами, отправка HTTP-запросов к API, отображение результатов
Веб-сервер	Nginx	Раздача статических файлов клиентского приложения, обратное проксирование запросов к API, точка входа для внешних подключений
Сервер приложения	NodeJS + Express	Реализация бизнес-логики, обработка REST-запросов, валидация данных, аутентификация пользователей.

Продолжение таблицы 2.16 – Основные компоненты архитектуры

Компонент	Технология	Назначение
База данных	PostgreSQL	Хранение данных приложения (пользователи, проекты, каталоги технологий, настройки движка)

Описание сетевого взаимодействия между компонентами архитектуры представлено в таблице 2.17.

Таблица 2.17 – Протоколы взаимодействия компонентов архитектуры

Протокол	Назначение
HTTPS/HTTP	Обмен данными между клиентом и Nginx; проксирование запросов от Nginx к серверу приложений
TCP	Взаимодействие сервера приложений с PostgreSQL

Выбранная архитектура обеспечивает четкое разделение ответственности между уровнями: клиент отвечает за представление данных, веб-сервер — за доставку фронтенда и маршрутизацию трафика, сервер приложений — за предметную логику, а база данных — за целостное и устойчивое хранение информации. Такой подход упрощает процесс сопровождения решения и ускоряет масштабирование: каждый слой можно развивать независимо без полной переработки всей системы.

2.5 Выводы по разделу

1. Разрабатываемое веб-приложение реализует ролевую модель доступа с двумя основными категориями пользователей: клиентом и администратором. Для каждой роли определён набор функций, соответствующий её задачам в системе.

2. Функциональные возможности приложения охватывают основные пользовательские сценарии: авторизацию, прохождение опроса по параметрам проекта, получение архитектурных рекомендаций, плана реализации и оценочной стоимости, экспорт диаграмм, а также административное управление пользователями, настройками движка и аналитикой системы.

3. Логическая модель базы данных включает 11 таблиц, отражающих ключевые сущности предметной области: пользователей, проекты, архитектурные планы, технологии, компоненты плана, настройки движка, административные действия и региональные данные. Для всех таблиц определены структура, типы полей и связи между сущностями, что обеспечивает полноту хранения данных и поддержку функциональных требований системы.

4. В качестве технологического стека выбраны Node.js и Express для серверной части, React и Vite для клиентского приложения, PostgreSQL в качестве системы управления базами данных, Nginx как веб-сервер и обратный прокси, а также Docker Compose для контейнеризации и развёртывания системы. Для работы с базой данных используется Drizzle, а для аутентификации и защиты API применяются JWT-механизмы.

5. Архитектура приложения построена по клиент-серверному принципу с использованием REST API, что обеспечивает разделение ответственности между уровнями системы, удобство сопровождения, возможность масштабирования и независимого развития отдельных компонентов.

3 Реализация веб-приложения

3.1 Программная платформа

Для серверной части проекта выбрана платформа Node.js (20 LTS) [9], ориентированная на асинхронную обработку запросов и эффективную работу с сетевыми операциями. В качестве веб-фреймворка используется Express.js (4.21.0) [10], обеспечивающий построение REST API [11], настройку маршрутов и подключение промежуточных обработчиков (middleware).

Клиентская часть реализована на библиотеке React (19.2.4) с компонентным подходом к построению интерфейса. Для разработки и сборки используется Vite [12], обеспечивающий быстрый запуск в режиме разработки и оптимизированную production-сборку. Для взаимодействия с сервером применяются HTTP-запросы через Axios [13], а маршрутизация в интерфейсе организована с помощью React Router.

3.2 Реляционная база данных PostgreSQL

В качестве системы управления базами данных в проекте используется PostgreSQL (16), обеспечивающая надежное хранение структурированных данных и поддержку ACID-транзакций. В базе данных хранятся сущности предметной области: пользователи, проекты, технологии, настройки движка, планы генерации, логи изменений системы и вспомогательные таблицы

Подключение к PostgreSQL в серверной части реализовано через Drizzle [14]. Данный инструмент обеспечивает:

- типобезопасное взаимодействия с базой данных;
 - удобное построение SQL-запросов и описание схемы таблиц на уровне программного кода;
 - поддержку миграций и согласованного управления базой данных
- Конфигурация подключения к базе данных представлена в листинге 3.1.

```
let pool = null;
let db = null;
function getPool() {
  const { Pool } = require("pg");

  pool = new Pool({
    connectionString: process.env.DATABASE_URL,
    ssl: process.env.DATABASE_SSL === "true" ? { rejectUnauthorized:
false } : false,
  });
  return pool;
}
module.exports = {
  getPool
}
```

Листинг 3.1 – Подключение к PostgreSQL через Drizzle

В листинге 3.1 показано создание единого экземпляра подключения к базе данных с использованием Drizzle и параметров, получаемых из конфигурации окружения. Это позволяет централизованно использовать механизм доступа к PostgreSQL во всех модулях серверной части приложения и обеспечивает единообразие работы с базой данных.

Пример безопасного параметризованного запроса, который осуществляет получение архитектурного плана конкретного пользователя по идентификатору плана, приведён в листинге 3.2.

```
const [row] = await db
  .select({
    id: planRuns.id,
    planId: planRuns.planId,
    inputPayload: planRuns.inputPayload,
    summary: planRuns.summary,
    recommendationPayload: planRuns.recommendationPayload,
    regionProfilePayload: planRuns.regionProfilePayload,
    roadmapPayload: planRuns.roadmapPayload,
    developmentPlanPayload: planRuns.developmentPlanPayload,
    costPayload: planRuns.costPayload,
    diagramPayload: planRuns.diagramPayload,
    drawioXml: planRuns.drawioXml,
    createdAt: planRuns.createdAt,
  })
  .from(planRuns)
  .where(and(eq(planRuns.userId,      userId),      eq(planRuns.planId,
planId)))
  .limit(1);
```

Листинг 3.2 – Пример безопасного параметризованного запроса

В данном запросе фильтрация выполняется по двум параметрам: идентификатору пользователя и идентификатору плана. Использование Drizzle позволяет формировать типобезопасные условия выборки и автоматически параметризовать передаваемые значения, что снижает вероятность ошибок и повышает безопасность взаимодействия с базой данных.

Структура базы данных в проекте формируется и поддерживается с помощью миграций Drizzle. Полный набор миграций и seed-данных представлен в каталоге backend/db/migrations.

3.3 Программные библиотеки

Для реализации функциональности приложения используются серверные и клиентские библиотеки.

Их выбор обусловлен требованиями к безопасности, удобству разработки, расширяемости и производительности.

В таблице 3.1 приведены ключевые библиотеки серверной части.

Таблица 3.1 – Ключевые библиотеки серверной части

Наименование	Версия	Назначение
express	5.2.1	Веб-фреймворк для создания серверной части приложения и реализации REST API
drizzle-orm	0.44.2	Библиотека для работы с PostgreSQL, описания схемы базы данных и построения типобезопасных запросов
pg	8.13.1	Драйвер подключения серверной части к базе данных PostgreSQL
dotenv	16.4.7	Загрузка переменных окружения из .env файлов в серверное приложение
express-oauth2-jwt-bearer	1.7.1	Проверка JWT-токенов и защита маршрутов API

Как следует из таблицы 3.1, основу серверной части составляют Express и Drizzle совместно с pg. Для безопасности применяется express-oauth2-jwt-bearer. В таблице 3.2 приведён перечень основных библиотек для клиентской части.

Таблица 3.2 – Ключевые библиотеки клиентской части

Наименование	Версия	Назначение
React	18.3.1	Библиотека для построения пользовательского интерфейса одностраничного веб-приложения
React-dom	18.3.1	Библиотека для отображения компонентов React в браузере
Auth0/auth0-react	2.3.0	Интеграция клиентского приложения с системой аутентификации Auth0
Vite	5.4.11	Инструмент сборки и локального запуска клиентского приложения
Vitejs/plugin-react	4.3.4	Плагин, обеспечивающий корректную работу Vite вместе с react
i18next	25.8.18	Ядро интернационализации
react-i18next	16.5.8	Интеграция i18n с React

Таким образом, клиентская часть построена на современном React-стеке с поддержкой маршрутизации и локализации.

3.4 Структура серверной части

Серверная часть приложения реализована на Node.js с использованием Express и организована по модульному принципу с разделением

ответственности между маршрутами, репозиториями, движком генерации, модулями работы с базой данных и вспомогательной инфраструктурой.

В состав серверной части входят следующие компоненты:

- src/routes - описание API-эндпоинтов;
- db/repositories – доступ к данным и реализация операций чтения, записи, обновления и удаления записей в базе данных;
- db – подключение к базе данных, описание схемы таблиц и миграции;
- engine – логика формирования архитектурных рекомендаций, расчёта стоимости, построения дорожной карты и генерации диаграмм;
- src – инициализация сервера, настройка маршрутизации и вспомогательные серверные модули;
- scripts – служебные сценарии для администрирования приложения;
- utils – общие функции, используемые различными частями серверной логики;

В таблице 3.3 приведён перечень основных директорий серверной части с указанием их назначения.

Таблица 3.3 – Директории серверной части

Директория	Назначение
src	Содержит исходный код серверной части, включая запуск приложения, настройку маршрутизации и основные серверные модули
src/routes	Содержит маршруты REST API, обеспечивающие обработку запросов от клиентской части
db	Содержит модули подключения к базе данных, описание схемы и другие компоненты, связанные с хранением данных
db/repositories	Содержит репозитории для работы с сущностями базы данных и реализации операций чтения, добавления, изменения и удаления записей
db/migrations	Содержит миграции базы данных, используемые для создания и изменения её структуры
engine	Содержит логику движка рекомендаций, включая формирование архитектурных решений, оценку стоимости, построение дорожной карты и генерацию диаграмм
scripts	Содержит служебные сценарии для администрирования и вспомогательных операций

Главная точка входа серверного приложения находится в src/server.js; настройка Express-приложения и подключение middleware выполняются в src/app.js. Такой подход обеспечивает удобство сопровождения, тестирования и расширения функциональности проекта.

3.5 Реализация функций

Реализация функциональности веб-приложения выполнена по архитектуре REST API: каждая прикладная функция представлена отдельным маршрутом, который принимает HTTP-запрос, при необходимости выполняет

проверку входных данных, передаёт управление соответствующему модулю серверной логики и возвращает структурированный JSON-ответ.

Серверная часть реализует следующие ключевые группы функций:

- получение анкеты проекта для последующего заполнения пользователем;
- аутентификация пользователя и синхронизация его профиля с серверной частью приложения;
- генерация архитектурного плана на основе введенных параметров проекта;
- получение, просмотр и удаление сохранённых архитектурных планов пользователя;
- получение аналитической информации о работе системы;
- административное управление пользователями;
- административное управление настройками движка рекомендаций;
- административное управление каталогом технологий.

Маршрут генерации архитектурного плана, представленный в листинге 3.3, принимает данные, введенные пользователем в анкете проекта, передаёт их в движок рекомендаций, сохраняет полученный результат в базе данных и возвращает сформированный архитектурный план вместе с сопутствующей информацией о его сохранении.

```
router.post("/generate", async (req, res, next) => {
  try {
    const engineSettings = await
engineSettingsRepository.getEngineSettings();
    const plan = generatePlan(req.body, engineSettings);
    const savedPlan = await
repository.saveGeneratedPlan(req.currentUser?.id, plan);
    const planWithTechnologies = {
      ...plan,
      technologies: savedPlan?.technologies || [],
    };

    res.status(200).json({
      plan: planWithTechnologies,
      savedPlan,
    });
  } catch (error) {
    next(error);
  }
});
```

Листинг 3.3 – Маршрут генерации архитектурного плана

В разрабатываемом приложении аутентификация пользователей выполняется с использованием внешнего сервиса Auth0, поэтому серверная часть не осуществляет самостоятельное формирование access- и refresh-токенов. Вместо этого защищённые маршруты выполняют проверку JWT-токена и, при успешной аутентификации, предоставляют доступ к данным пользователя и функциям системы.

Функция получения анкеты проекта обеспечивает передачу на клиентскую сторону структуры опроса, необходимой для формирования интерфейса ввода исходных параметров. Маршрут, представленный в листинге 3.4, возвращает данные анкеты в формате JSON, включая идентификаторы полей, их типы, признаки обязательности, наборы допустимых значений и поясняющие подсказки. Такой формат позволяет использовать полученные данные непосредственно для отображения формы в клиентском приложении.

```
const { Router } = require("express");

const questionnaire = [
  {
    id: "projectName",
    label: "Project name",
    type: "text",
    placeholder: "Clinic CRM",
    required: true,
    helpText: "Used to label the generated plan and downloadable files.",
  },
  {
    id: "projectStage",
    label: "Project stage",
    type: "select",
    required: true,
    options: ["idea", "prototype", "mvp", "growing-product"],
    helpText: "Helps balance simplicity against future scalability.",
  },
  {
    id: "monthlyBudget",
    label: "Monthly infrastructure budget (USD)",
    type: "number",
    required: true,
    min: 0,
    helpText: "Constrains architecture complexity and provider choice.",
  },
];

const router = Router();

router.get("/", (req, res) => {
  res.status(200).json({
    questionnaire,
  });
});

module.exports = router;
```

Листинг 3.4 – Маршрут для получения данных анкеты

Маршрут использует предварительную аутентификацию пользователя и выполняет обращение только к тем данным, которые принадлежат текущей учётной записи. Ответ возвращается в формате JSON и содержит как сам сохранённый архитектурный план, так и служебные сведения о моменте его создания.

Одной из ключевых функций системы является получение ранее сохранённого архитектурного плана по его идентификатору. Для этого используется проверка по двум параметрам: идентификатору текущего пользователя и идентификатору плана. Такой подход исключает возможность доступа пользователя к чужим архитектурным планам и обеспечивает корректную выборку нужных данных из базы. Реализация маршрута представлена в листинге 3.5.

```
router.get("/:planId", async (req, res, next) => {
  try {
    const savedPlan = await repository.getUserPlanByPlanId(
      req.currentUser?.id,
      req.params.planId
    );

    if (!savedPlan) {
      return res.status(404).json({
        error: "Project not found for the current user.",
      });
    }

    return res.status(200).json({
      plan: savedPlan.plan,
      savedPlan: {
        id: savedPlan.id,
        createdAt: savedPlan.createdAt,
      },
    });
  } catch (error) {
    return next(error);
  }
});
```

Листинг 3.5 – Маршрут для получения архитектурного плана

При обращении к базе данных используется автоматическая параметризация запросов средствами Drizzle, что снижает риск SQL-инъекций и обеспечивает безопасную передачу пользовательских данных в условия выборки и операции изменения записей.

Маршрут добавления новой технологии, представленный в листинге 3.6, выполняет приём и обработку данных, переданных администратором, проверку корректности идентификатора категории и её активности, сохранение новой технологии в каталог, а также регистрацию выполненного действия в журнале административных изменений. Такой подход позволяет обеспечить целостность данных и контроль над изменениями, вносимыми в систему.

```

router.post("/technologies", async (req, res, next) => {
  try {
    const technology = await technologyRepository.createTechnology(
      req.body || {},
      req.currentUser?.id
    );

    res.status(201).json({
      technology,
    });
  } catch (error) {
    next(error);
  }
});

```

Листинг 3.6 – Маршрут добавления новой технологии

Одной из ключевых административных функций системы является настройка параметров движка рекомендаций. Изменение этих параметров позволяет корректировать правила формирования архитектурных решений, коэффициенты оценки и иные значения, влияющие на итоговый результат генерации плана.

Маршрут, представленный в листинге 3.7, предназначен для сохранения обновлённых настроек движка. Доступ к нему имеют только пользователи с ролью администратора. После получения новых параметров маршрут передаёт их в соответствующий модуль серверной логики, который выполняет нормализацию данных, сохраняет их в базе данных и фиксирует факт изменения в журнале административных действий.

```

router.patch("/settings/engine", async (req, res, next) => {
  try {
    const settingsRecord = await
engineSettingsRepository.saveEngineSettings(
      req.currentUser?.id,
      req.body?.settings || {}
    );

    res.status(200).json(settingsRecord);
  } catch (error) {
    next(error);
  }
});

```

Листинг 3.7 – Маршрут для сохранения обновлённых настроек движка

Администратор может изменять роль пользователя в системе. Внесённые изменения сохраняются в таблице users и используются для разграничения прав доступа к административным и пользовательским функциям приложения.

Для динамического управления правами доступа реализован endpoint, обновляющий роль выбранного пользователя. Все изменения сохраняются в базе

данных и дополнительно фиксируются в журнале административных действий, что позволяет контролировать историю изменения пользовательских полномочий. Реализация маршрута представлена в листинге 3.9.

```
router.patch("/users/:userId/role", async (req, res, next) => {
  try {
    const targetUserId = parseUserId(req.params.userId);
    if (targetUserId === null) {
      return res.status(400).json({
        error: "User id must be a valid integer.",
      });
    }
    const user = await userRepository.updateUserRole(
      targetUserId,
      String(req.body?.role || ""),
      req.currentUser?.id
    );
    return res.status(200).json({
      user,
    });
  } catch (error) {
    return next(error);
  }
});
```

Листинг 3.9 – Маршрут для изменения пользовательских полномочий

Администратор управляет пользователями, настройками движка рекомендаций и каталогом технологий через административный модуль admin. Данный модуль объединяет функции, связанные с сопровождением системы, настройкой параметров генерации архитектурных решений и контролем доступных технологических компонентов.

Реализация одного из административных маршрутов представлена в листинге 3.10.

```
router.get("/technologies", async (req, res, next) => {
  try {
    const includeInactive =
      String(req.query.includeInactive || "").toLowerCase() ===
      "true";
    const technologies = await technologyRepository.listTechnologies({
      includeInactive,
    });
    res.status(200).json({
      technologies,
    });
  } catch (error) {
    next(error);
  }
});
```

Листинг 3.10 – Реализация маршрута для получения технологий

Для панели администратора в приложении реализован агрегирующий модуль аналитики, предназначенный для получения сводной информации о работе системы, активности пользователей и характеристиках сгенерированных архитектурных решений.

Административный дашборд собирает данные из нескольких таблиц базы данных, включая `users` (общее количество пользователей и распределение по ролям), `plan_runs` (количество созданных архитектурных планов, сведения о выбранных архитектурных стилях, моделях развёртывания и регионах), `plan_run_technologies` и `technologies` (наиболее часто используемые технологии), а также другие связанные сущности, участвующие в формировании аналитических показателей.

Реализация маршрута представлена в листинге 3.11.

```
router.get("/analytics/overview", async (req, res, next) => {
  try {
    const overview = await analyticsRepository.getOverview();

    res.status(200).json(overview);
  } catch (error) {
    next(error);
  }
});
```

Листинг 3.11 – Маршрут для получения аналитики системы

Таким образом, реализация функций в `ArchitecturePlanner` охватывает полный цикл основных пользовательских и административных сценариев: от аутентификации и получения анкеты проекта до генерации архитектурного плана, сохранения результатов, управления каталогом технологий, настройки движка рекомендаций и получения аналитической информации о работе системы.

Использование REST-маршрутов, модульной серверной структуры, механизмов аутентификации, централизованного доступа к данным и унифицированного формата JSON-ответов обеспечивает расширяемость, сопровождаемость и устойчивость прикладной части разрабатываемого веб-приложения.

3.6 Структура клиентской части

Клиентская часть разработана на `React` и организована по модульному принципу. Структура ориентирована на разделение страниц, переиспользуемых компонентов и прикладной логики. Директории клиентской части представлены в таблице 3.4.

Таблица 3.4 – Директории клиентской части

Директория	Назначение
<code>src</code>	Содержит исходный код клиентской части приложения

Продолжение таблицы 3.4 – Директории клиентской части

Директория	Назначение
src/components	Содержит пользовательские интерфейсные компоненты, используемые для построения страниц и отдельных функциональных блоков
src/config	Содержит конфигурационные параметры клиентского приложения
src/constants	Содержит константы, используемые в логике работы интерфейса и отдельных модулей
src/hooks	Содержит пользовательские React-хуки, реализующие повторно используемую клиентскую логику
src/utils	Содержит вспомогательные функции для обработки данных, форматирования и экспорта результатов
dist	Содержит собранную версию клиентского приложения, подготовленную для развёртывания

Клиентская навигация в приложении реализована без использования react-router-dom: переключение между основными представлениями интерфейса выполняется внутри корневого компонента App.jsx на основе текущего состояния приложения. Сетевое взаимодействие с серверной частью реализовано с использованием встроенного fetch API. Основная логика выполнения HTTP-запросов сосредоточена в пользовательском хуке usePlannerApp, а базовый адрес серверного API вынесен в отдельный конфигурационный модуль config/api.js.

3.7 Настройка конфигурации Docker

```

services:
  db:
    build:
      context: .
      dockerfile: db/Dockerfile
    environment:
      POSTGRES_DB: architecture_planner
      POSTGRES_USER: architectureplanner
      POSTGRES_PASSWORD: architectureplanner
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build:
      context: ./backend
    depends_on:
      - db
    env_file:
      - ./backend/.env

  nginx:
    build:
      context: .
      dockerfile: nginx/Dockerfile
    depends_on:

```

```

    - backend
  ports:
    - "80:80"

volumes:
  postgres_data:

```

Листинг 3.12 – Конфигурация Docker Compose

В приведённой конфигурации описаны три основных сервиса системы: база данных PostgreSQL, серверная часть приложения и веб-сервер Nginx. Контейнер backend зависит от сервиса базы данных, а nginx используется как внешняя точка доступа к приложению. Для сохранения данных PostgreSQL используется именованный том postgres_data, что обеспечивает их сохранность при перезапуске контейнеров.

Сервис базы данных использует volume для постоянного хранения данных, а переменные окружения обеспечивают правильную настройку базы данных.

В листинге 3.13 представлен файл конфигурации Docker для серверной части. Данный Dockerfile предназначен для сборки образа серверной части приложения на основе официального образа node:20-alpine. Выбор дистрибутива Alpine обусловлен минимальным размером финального образа.

```

FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY db ./db
COPY engine ./engine
COPY scripts ./scripts
COPY src ./src
EXPOSE 4000
CMD ["npm", "start"]

```

Листинг 3.13 – Dockerfile серверной части приложения

В процессе сборки в контейнер копируются файлы зависимостей, устанавливаются только production-пакеты, после чего добавляются исходные модули серверного приложения. В качестве стартовой команды используется prn start, запускающая сервер Express внутри контейнера.

3.8 Настройка конфигурации Nginx

Nginx используется как единая точка входа в систему: раздает статические файлы клиентского приложения и проксирует API-запросы на backend.

В листинге 3.14 представлена конфигурация Nginx.

```

server {
    listen 80;
    server_name _;
    return 301 https://$host:20532$request_uri;
}
server {
    listen 20532 ssl;
    server_name _;
    ssl_certificate /etc/nginx/certs/fullchain.pem;
    ssl_certificate_key /etc/nginx/certs/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_session_timeout 1d;
    ssl_session_cache shared:SSL:10m;
    root /usr/share/nginx/html;
    index index.html;
    location /api/ {
        proxy_pass http://backend:4000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}

```

Листинг 3.14 – Конфигурация Nginx

Данная конфигурация Nginx обеспечивает перенаправление входящих HTTP-запросов на защищённое HTTPS-соединение и выступает точкой входа для клиентской и серверной частей приложения. Запросы к маршрутам /api/ проксируются на сервер backend, а все остальные обращения обрабатываются как запросы к статическим файлам клиентского приложения.

3.9 Выводы по разделу

1. Реализация функций в проекте ArchitecturePlanner выполнена на основе REST-архитектуры с разделением маршрутов, репозиториев, модулей работы с базой данных и движка рекомендаций.

2. Описаны ключевые пользовательские и административные функции системы, включая аутентификацию, получение анкеты проекта, генерацию архитектурного плана, сохранение и просмотр проектов, управление пользователями, каталогом технологий, настройками движка и аналитикой.

3. Определён единый формат обмена данными в JSON, используемый при взаимодействии клиентской и серверной частей приложения.

4. Клиентская часть структурирована по модульному принципу и включает компоненты интерфейса, конфигурационные модули, константы, пользовательские хуки и вспомогательные функции, что упрощает сопровождение и развитие приложения.

5. Для развёртывания системы используется контейнеризация Docker и конфигурация Docker Compose, включающая три основных сервиса: nginx, backend и db, что обеспечивает воспроизводимость среды и упрощает запуск приложения.

6. Nginx настроен как точка входа в систему, выполняющая перенаправление на HTTPS, раздачу статических файлов клиентского приложения и проксирование запросов к серверному API, что повышает удобство эксплуатации, устойчивость и безопасность веб-приложения.

4 Тестирование веб-приложения

4.1 Функциональное тестирование

Для проверки корректности работы веб-приложения было выполнено функциональное тестирование основных пользовательских и административных сценариев. Проверялись операции аутентификации, получения анкеты проекта, генерации архитектурного плана, сохранения и просмотра проектов, экспорта диаграмм, а также административные функции управления пользователями, каталогом технологий, настройками движка и аналитикой системы.

Результаты функционального тестирования приведены в таблице 4.1.

Таблица 4.1 – Результаты функционального тестирования

№	Функция	Описание теста	Ожидаемый результат	Фактический результат
1	Проверка доступности API	GET /api/health	Возврат информации о доступности серверной части	Код 200, сервер доступен
2	Получение анкеты проекта	GET /api/questionnaire	Возврат структуры анкеты для заполнения пользователем	Код 200, данные получены
3	Получение профиля пользователя	GET /api/auth/me с Bearer token	Возврат данных текущего пользователя	Код 200, профиль получен
4	Синхронизация профиля пользователя	PATCH /api/auth/profile с корректными данными профиля	Обновление данных пользователя в системе	Код 200, данные обновлены
5	Генерация архитектурного плана	POST /api/plans/generate с корректно заполненной анкетой	Возврат архитектурного плана, оценки стоимости и диаграммы	Код 200, план успешно сгенерирован
6	Получение списка проектов пользователя	GET /api/plans с Bearer token	Возврат списка ранее сохранённых проектов пользователя	Код 200, список получен
7	Получение сохранённого плана	GET /api/plans/:planId с корректным идентификатором плана	Возврат полного сохранённого архитектурного плана	Код 200, данные получены
8	Получение списка пользователей	GET /api/admin/users с Bearer token администратора	Возврат списка зарегистрированных пользователей	Код 200, данные получены
9	Получение настроек движка рекомендаций	GET /api/admin/settings/engine с Bearer token администратора	Возврат текущих параметров движка рекомендаций	Код 200, данные получены

Продолжение таблицы 4.1 – Результаты функционального тестирования

№	Функция	Описание теста	Ожидаемый результат	Фактический результат
10	Получение аналитики системы	GET /api/admin/analytics/overview с Bearer token администратора	Возврат агрегированных метрик по работе системы	Код 200, данные получены

Проведенное функциональное тестирование подтвердило корректную работу ключевых функций в пользовательской и административной частях приложения.

4.2 Автоматизированное тестирование

Для повышения надёжности проверки был реализован отдельный набор автоматизированных функциональных тестов серверной части приложения. Тесты вынесены в независимую директорию C:\Users\Manmade\Desktop\ArchitecturePlanner\test, что исключает их влияние на основной код проекта.

Автоматизированное тестирование выполняется скриптом functional-tests.mjs, который:

- последовательно проверяет публичные REST-маршруты и ключевые серверные модули приложения;
- фиксирует код ответа, время выполнения и результат каждого шага;
- формирует итоговый отчёт в формате JSON (functional-report.json).

В ходе автоматизированного тестирования были проверены доступность API, получение анкеты проекта, корректность генерации архитектурного плана, сохранение и чтение данных из базы PostgreSQL, работа с каталогом технологий, сохранение настроек движка рекомендаций и получение агрегированной аналитики системы.

Результаты автоматизированного тестирования представлены в таблице 4.2.

Таблица 4.2 – Результаты автоматизированного тестирования

№	Тест	Описание теста	Результат
1	testApiHealth	Проверка маршрута доступности /api/health	200: успешно
2	testQuestionnaireEndpoint	Проверка получения анкеты проекта	200: успешно
3	testDatabaseConnection	Проверка соединения с PostgreSQL	200: успешно
4	testGeneratePlan	Проверка генерации архитектурного плана	200: успешно
5	testSaveGeneratedPlan	Проверка сохранения сгенерированного плана	200: успешно
6	testListUserPlans	Проверка получения списка проектов пользователя	200: успешно
7	testGetUserPlan	Проверка получения сохранённого плана по идентификатору	200: успешно

Продолжение таблицы 4.2 – Результаты автоматизированного тестирования

№	Тест	Описание теста	Результат
8	testCreateTechnology	Проверка добавления технологии в каталог	200: успешно
9	testUpdateEngineSettings	Проверка сохранения настроек движка рекомендаций	200: успешно
10	testAnalyticsOverview	Проверка получения агрегированной аналитики системы	200: успешно

Таким образом, автоматизированные тесты подтвердили корректную работу основных функций серверной части приложения и обеспечили воспроизводимую проверку после внесения изменений в проект. По результатам выполненного прогона все 10 тестов завершились успешно.

4.3 Нагрузочное тестирование

Для оценки устойчивости и производительности серверной части приложения было выполнено нагрузочное тестирование. Тестирование проводилось с использованием отдельного сценария load-test.mjs, размещённого в директории C:\Users\Manmade\Desktop\ArchitecturePlanner\test.

Параметры профиля нагрузки:

- количество параллельных виртуальных пользователей: 25;
- длительность теста: 60 секунд;
- тип нагрузки: постоянная;
- проверяемые

маршруты:

/api/questionnaire, /api/plans/generate, /api/admin/analytics/overview.

В процессе тестирования фиксировались общее число запросов, среднее время отклика, доля успешных запросов, доля ошибок и пиковые значения времени ответа. Итоги нагрузочного тестирования приведены в таблице 4.3.

Таблица 4.3 – Результаты нагрузочного тестирования

Параметр	Значение
Количество пользователей	25
Длительность теста	60 секунд
Тип нагрузки	Постоянная
Общее количество запросов	10394
Среднее время отклика	144,5 мс
Максимальное время отклика	847,4 мс
Доля успешных запросов	100%
Доля ошибок	0%
Пропускная способность	173,2 запр./с

Нагрузочное тестирование показало, что приложение сохраняет работоспособность при выбранной параллельной нагрузке и демонстрирует стабильные показатели времени отклика. По результатам теста ошибки выполнения запросов отсутствовали, а серверная часть корректно обрабатывала как операции

генерации архитектурного плана, так и запросы на получение аналитических и справочных данных.

4.4 Выводы по разделу

1. В ходе тестирования веб-приложения были проведены функциональные, автоматизированные и нагрузочные проверки.

2. Функциональное тестирование подтвердило корректную работу основных пользовательских и административных сценариев, включая получение анкеты проекта, генерацию архитектурного плана, сохранение проектов, управление технологиями, настройками движка и аналитикой.

3. Автоматизированное тестирование реализовано в виде отдельного набора тестов и обеспечивает воспроизводимую проверку серверной части приложения после внесения изменений.

4. Нагрузочное тестирование подтвердило стабильную работу серверной части при параллельной обработке пользовательских запросов в рамках выбранного профиля нагрузки.

5. Суммарные результаты тестирования показывают, что разработанное веб-приложение соответствует поставленным функциональным требованиям и может использоваться в качестве основы для дальнейшей эксплуатации и развития.

5. Руководство пользователя

5.1 Авторизация

Для регистрации пользователю необходимо нажать на кнопку выпадающего меню в правом верхнем углу страницы и в появившемся меню нажать кнопку «Регистрация». Страница регистрации представлена на рисунке 5.1.

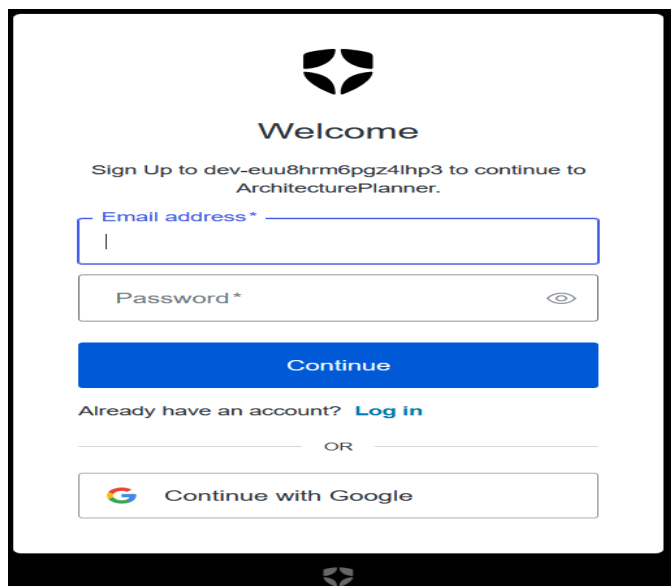


Рисунок 5.1 – Страница регистрации

Далее пользователь заполняет предложенные поля и нажимает на кнопку «Continue», после чего его данные сохраняются в базе данных и он получает доступ к пользовательскому функционалу. Если пользователь уже зарегистрирован в системе и хочет войти в свой аккаунт – ему необходимо нажать на кнопку «Войти» в ранее описанном выпадающем меню, после чего откроется страница входа, представленная на рисунке 5.2.

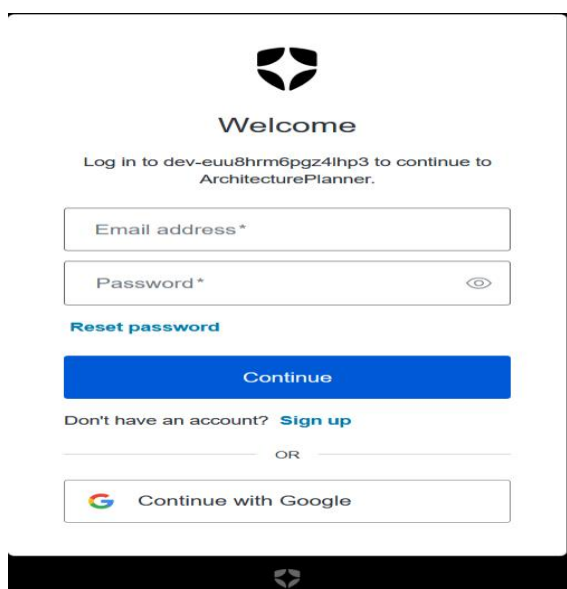


Рисунок 5.2 – Страница входа

После заполнения предложенных полей корректными данными пользователь получает доступ к функциям системы согласно роли и своим сохранённым данным.

5.2. Прохождение опроса о продукте

Вне зависимости от роли пользователь имеет возможность заполнить данные о программном продукте, для которого необходимо сгенерировать архитектуру. Данные собираются системой в формате опроса, что продемонстрировано на рисунке 5.3.

ПРОФЕССИОНАЛЬНАЯ СРЕДА ДЛЯ ПРОЕКТИРОВАНИЯ АРХИТЕКТУРЫ

ArchitecturePlanner

Планировщик

Детерминированное планирование архитектуры для стартапов и небольших компаний. Проектные ограничения, анализируйте инфраструктурные рекомендации и храните историю проектов для каждого авторизованного пользователя.

Анкета проекта

Эти ответы напрямую влияют на rule engine. В логике рекомендаций ИИ не используется.

Название проекта

CRM для клиники

Используется для подписи плана и скачиваемых файлов.

Стадия проекта

Идея

Помогает сбалансировать простоту решения и будущую масштабируемость.

Тип бизнеса

SaaS

Рисунок 5.3 – Главная страница приложения

Опрос о продукте разбит на некоторое количество вопросов с целью собрать максимальное количество информации с максимальной ценностью для системы. Помимо основных вопросов о программном продукте пользователь также может включить в желаемую архитектуру дополнительные функции, что продемонстрировано на рисунке 5.4.

Определяет, сколько поверхностей доставки должна поддерживать система.

Ключевые функции

☒ Аутентификация	☐ Платежи
☐ Загрузка файлов	☐ Поиск
☐ Админ-панель	☐ Уведомления

Каждая выбранная функция добавляет архитектурные компоненты.

Рисунок 5.4 – Включение дополнительных функций в генерируемую архитектуру

Помимо выбора дополнительных функций пользователю также предоставляется возможность включать или не включать в генерируемую архитектуру дополнительные модули, что продемонстрировано на рисунке 5.5.

Нужны realtime-возможности Добавляет websocket- или pub/sub-компоненты, когда они необходимы. ☐

Рисунок 5.5 – Включение дополнительных модулей в генерацию архитектуры

5.3 Генерация архитектуры

После ввода наименования проекта, для которого генерируется архитектура, заполнения информации в предлагаемых системой полях (стадия развития проекта, тип бизнеса, целевой регион, предпочтения по развёртыванию, оценка месячной аудитории, месячный бюджет и т.д.), становится активной кнопка «Сгенерировать архитектурный план», которая расположена в нижней левой части главной страницы. При нажатии на эту кнопку пользователь получает сгенерированную архитектуру вместе с кратким резюме рекомендаций, компонентами архитектуры, подходящими технологиями, оценкой стоимости развёртывания и иными метриками, а также возможность скачать диаграмму сгенерированной архитектуры в одном из двух предложенных форматов: SVG или DRAWIO (рисунок 5.6).

Региональные заметки

- Сильная поддержка управляемых облачных сервисов.
- Базовый регион стоимости для оценок.

Рисунок 5.6 – Кнопки для скачивания диаграмм архитектуры

Общее представление блока «Сгенерированный результат», располагающегося в правой части страницы и хранящего в себе все вышеописанные компоненты показано на рисунке 5.7.

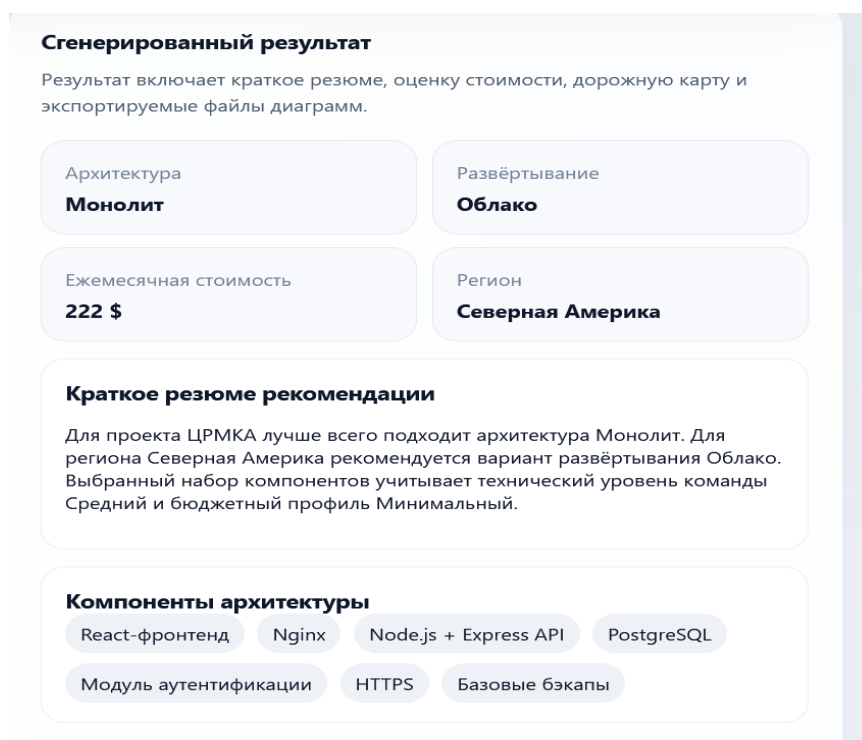


Рисунок 5.7 – Сгенерированная архитектура проекта

5.4 Личный кабинет

На странице личного кабинета пользователь просматривает информацию о своём профиле, а также собственную библиотеку проектов – сгенерированные пользователем проекты, сохранённые в системе. Данная страница показана на рисунке 5.8.

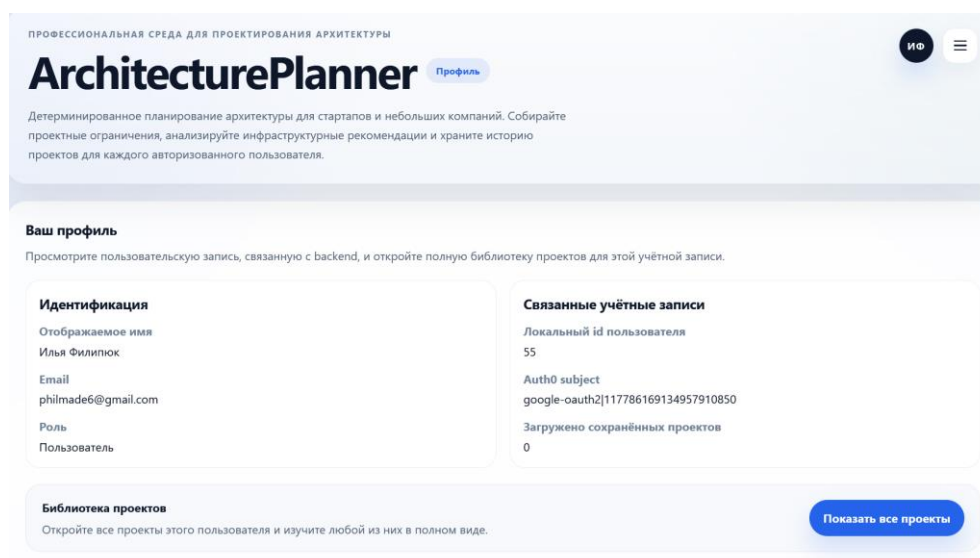


Рисунок 5.8 – Личный кабинет пользователя

Как было описано выше, на странице своего личного кабинета пользователь может просматривать все проекты, которые были ранее им сгенерированы и сохранены в системе. Для этого ему необходимо нажать кнопку «Показать все проекты», которая расположена в правом нижнем углу станицы. Страница проектов показана на рисунке 5.9.

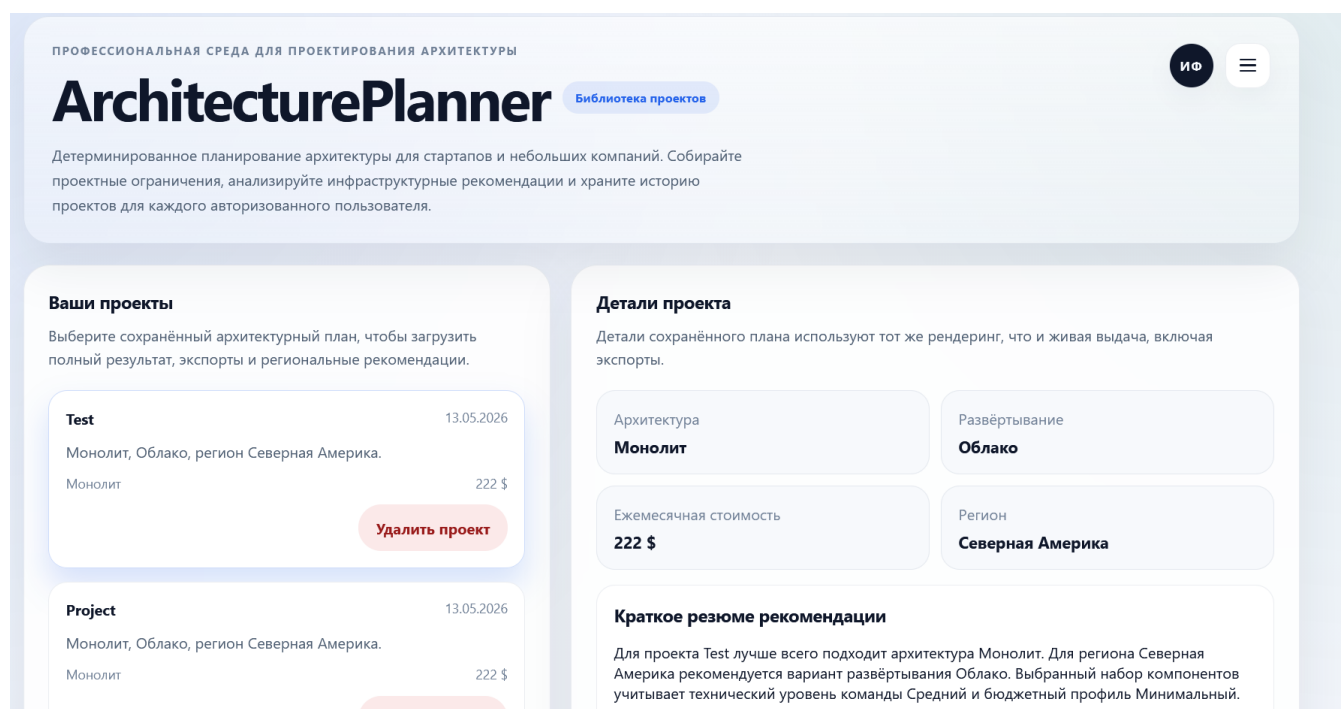


Рисунок 5.9 – Библиотека проектов

5.5 Панель администратора

Администратор после входа под учётной записью с соответствующей ролью получает доступ к административным маршрутам системы: просмотру аналитики, управлению пользователями, настройками движка рекомендаций и каталогом технологий. Административная панель предназначена для сопровождения работы приложения, контроля пользовательской активности и корректировки параметров, влияющих на формирование архитектурных рекомендаций.

На панели аналитики администратор видит ключевые показатели системы в виде карточек с числовыми значениями, включая общее количество пользователей, число администраторов, количество сгенерированных архитектурных планов и среднюю оценочную стоимость развёртывания проектов. Эти показатели позволяют оперативно оценивать текущее состояние системы и характер её использования.

Ниже располагаются аналитические блоки, отражающие наиболее популярные архитектурные стили, модели развёртывания, используемые технологии, целевые регионы и типы проектов. Дополнительно отображаются наиболее распространённые комбинации архитектурных компонентов, динамика создания планов и журнал недавних административных действий. Такой подход позволяет администратору отслеживать структуру пользовательских запросов, выявлять наиболее востребованные варианты архитектурных решений и оценивать особенности эксплуатации системы.

При отсутствии достаточного объёма данных отдельные аналитические блоки могут содержать ограниченное число записей, однако сама панель сохраняет свою информативность за счёт отображения агрегированных показателей и общей статистики использования приложения. Панель аналитики представлена на рисунке 5.10.

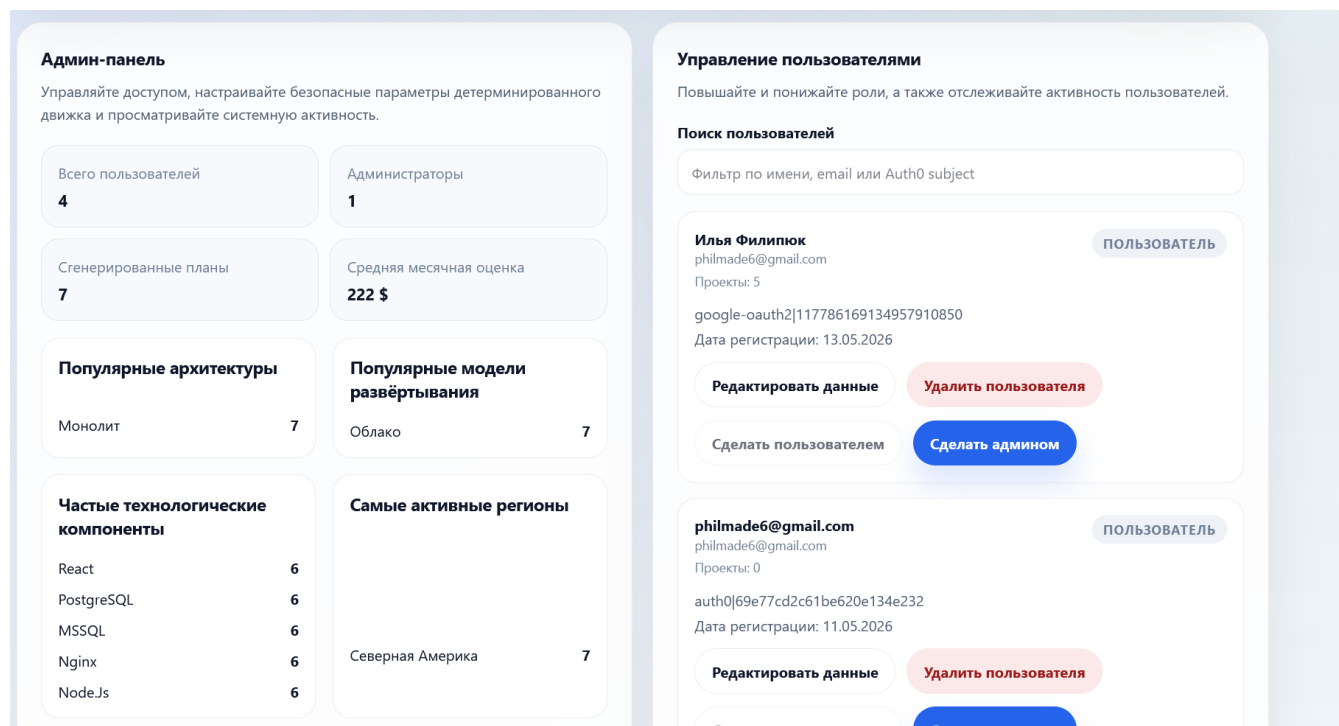


Рисунок 5.10 – Панель аналитики

Также, как было сказано выше, в панели администратора также присутствует блок настроек движка генерации, который позволяет администратору управлять алгоритмом генерации и его параметрами, а также блок управления каталогом технологий. Два данных блока панели администратора представлены на рисунке 5.11.

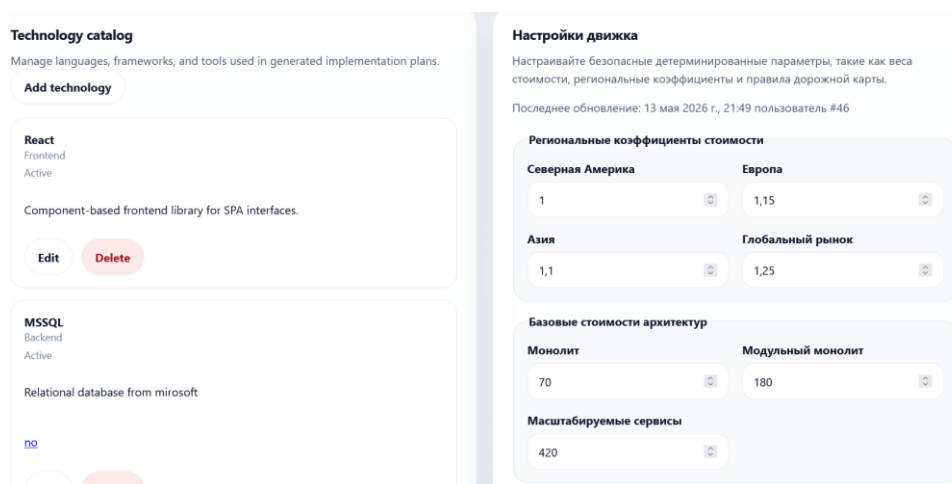


Рисунок 5.11 – Блоки управления движком

5.6 Развёртывание и проверка работоспособности

Типовой запуск ArchitecturePlanner выполняется с использованием Docker Compose.

Архитектура развёртывания включает три сервиса:

- db: сервис базы данных PostgreSQL 16;
- backend: серверное приложение на NodeJS и Express, работающее внутри контейнерной сети на порту 4000;
- nginx: веб-сервер, ответственный за публикацию клиентской части приложения, а также обратное проксирование запросов к серверному API.

Данные базы сохраняются в именованном томе postgres_data, что обеспечивает их сохранность между перезапусками контейнеров.

Подготовка окружения включает следующие действия:

- клонировать репозиторий проекта;
- создать и настроить файл backend/.env, указав параметры подключения к базе данных, а также необходимые переменные окружения для работы сервиса авторизации;
- заполнить клиентские переменные окружения в frontend/.env, указав параметры интеграции с Auth0;
- разместить файлы fullchain.pem и privkey.pem в каталоге nginx/certs;
- при необходимости сгенерировать самоподписанные сертификаты с помощью сценария nginx/generate-dev-certs.ps1.

При запуске через Docker Compose отдельная предварительная сборка клиентской части не требуется, поскольку она выполняется автоматически внутри nginx/Dockerfile. Инициализация структуры базы данных также выполняется автоматически при первом запуске контейнера db, так как схема загружается из файла backend/db/schema.sql. Если используется уже существующий том с данными от более ранней версии проекта, дополнительные изменения необходимо применить вручную из каталога backend/db/migrations.

Для запуска проекта из корневого каталога следует выполнить команду `docker compose up -d --build`. После запуска необходимо убедиться, что контейнер базы данных успешно проходит healthcheck, а сервисы backend и nginx работают без ошибок. Доступ к приложению в типовой конфигурации осуществляется по адресу `https://localhost:20532`, при этом обращения на `http://localhost:80` автоматически перенаправляются на защищённое соединение.

Проверка работоспособности включает следующие действия:

- главная страница приложения успешно открывается в браузере;
- анкета проекта загружается корректно и доступна для заполнения;
- генерация архитектурного плана выполняется без ошибок;
- после входа в систему пользователь получает доступ к профилю и сохранённым проектам;
- сформированный план содержит рекомендации, оценку стоимости, дорожную карту и диаграмму;

- пользователь с ролью «Администратор» получает доступ к панели администратора, аналитике, настройка движка и каталогу технологий.

Соблюдение указанной последовательности действий обеспечивает воспроизводимое развёртывание приложения в контейнеризированной среде и снижает вероятность ошибок конфигурации при переносе проекта на другой компьютер или сервер.

5.7 Выводы по разделу

1. В веб-приложении реализован доступ к функциям системы с разделением на три состояния использования: неавторизованный пользователь, авторизованный пользователь и администратор. Эти уровни отличаются составом доступных действий и требованиями к прохождению аутентификации.

2. Неавторизованный пользователь может открывать главную страницу приложения и заполнять анкету проекта, однако для выполнения защищённых действий, связанных с сохранением результатов и доступом к персональным данным, требуется вход в систему.

3. Авторизованный пользователь получает доступ к генерации и сохранению архитектурных планов, просмотру профиля, работе с библиотекой собственных проектов и повторному открытию ранее сформированных результатов.

4. Администратор обладает полномочиями по управлению пользователями, просмотру аналитики системы, редактированию параметров движка рекомендаций и ведению каталога технологий, используемого при формировании архитектурных решений.

5. Пользовательский интерфейс приложения построен таким образом, чтобы основные сценарии работы выполнялись последовательно и предсказуемо, а иллюстративные экранные формы в руководстве сопровождали описание действий пользователя на каждом этапе взаимодействия с системой.

Заключение

1. В результате выполнения курсового проекта разработано веб-приложение, предназначенное для автоматизации первичного архитектурного планирования программных систем. В приложении реализовано разграничение доступа по трём состояниям использования системы: неавторизованный пользователь, авторизованный пользователь и администратор. Для каждого уровня определён собственный набор доступных функций и ограничений.

2. В системе реализованы 8 ключевых функций, охватывающих основной функционал приложения: авторизация, прохождение анкеты проекта, генерация архитектурного плана, экспорт диаграммы, управление пользователями, управление каталогом технологий, настройка движка рекомендаций и получение аналитики системы. Реализованный функционал обеспечивает как пользовательские сценарии получения архитектурных рекомендаций, так и административные сценарии сопровождения системы.

3. База данных приложения включает 11 таблиц: `users`, `admin_audit_log`, `engine_settings`, `generated_plans`, `region_data_cache`, `plan_runs`, `plan_run_technologies`, `plan_components`, `projects`, `technologies`, `technology_categories`. Указанный набор сущностей обеспечивает хранение сведений о пользователях, проектах, сгенерированных архитектурных планах, компонентах, технологиях, настройках движка рекомендаций, административных действиях и региональных особенностях развёртывания.

4. Архитектура системы носит клиент-серверный характер. Клиентская часть реализована на React с использованием Vite. Серверная часть выполнена на платформе Node.js с использованием фреймворка Express, а доступ к PostgreSQL организован посредством Drizzle. Аутентификация пользователей обеспечивается через Auth0 и проверку JWT-токенов. Развёртывание приложения выполняется с помощью Docker Compose и Nginx, который используется как обратный прокси и сервер статических файлов. Выбранная архитектура обеспечивает разделение уровней представления, бизнес-логики и хранения данных, а также создаёт основу для дальнейшего масштабирования системы.

5. Объём программного кода прикладных частей клиента и сервера составляет 7349 строк. Из них 4002 строки приходятся на клиентскую часть (`frontend/src`, файлы `.js` и `.jsx`), а 3347 строк — на серверную часть (`backend/src`, `backend/db`, `backend/engine`, `backend/scripts`, файлы `.js`). Таким образом, объём разработанного программного обеспечения составляет порядка 7300 авторских строк кода.

6. Для обеспечения качества разработанного веб-приложения были проведены функциональное, автоматизированное и нагрузочное тестирование. Результаты выполненных проверок подтвердили корректную работу основных пользовательских и административных сценариев, воспроизводимость автоматизированной проверки серверной части и устойчивость приложения при параллельной обработке запросов в рамках выбранного профиля нагрузки.

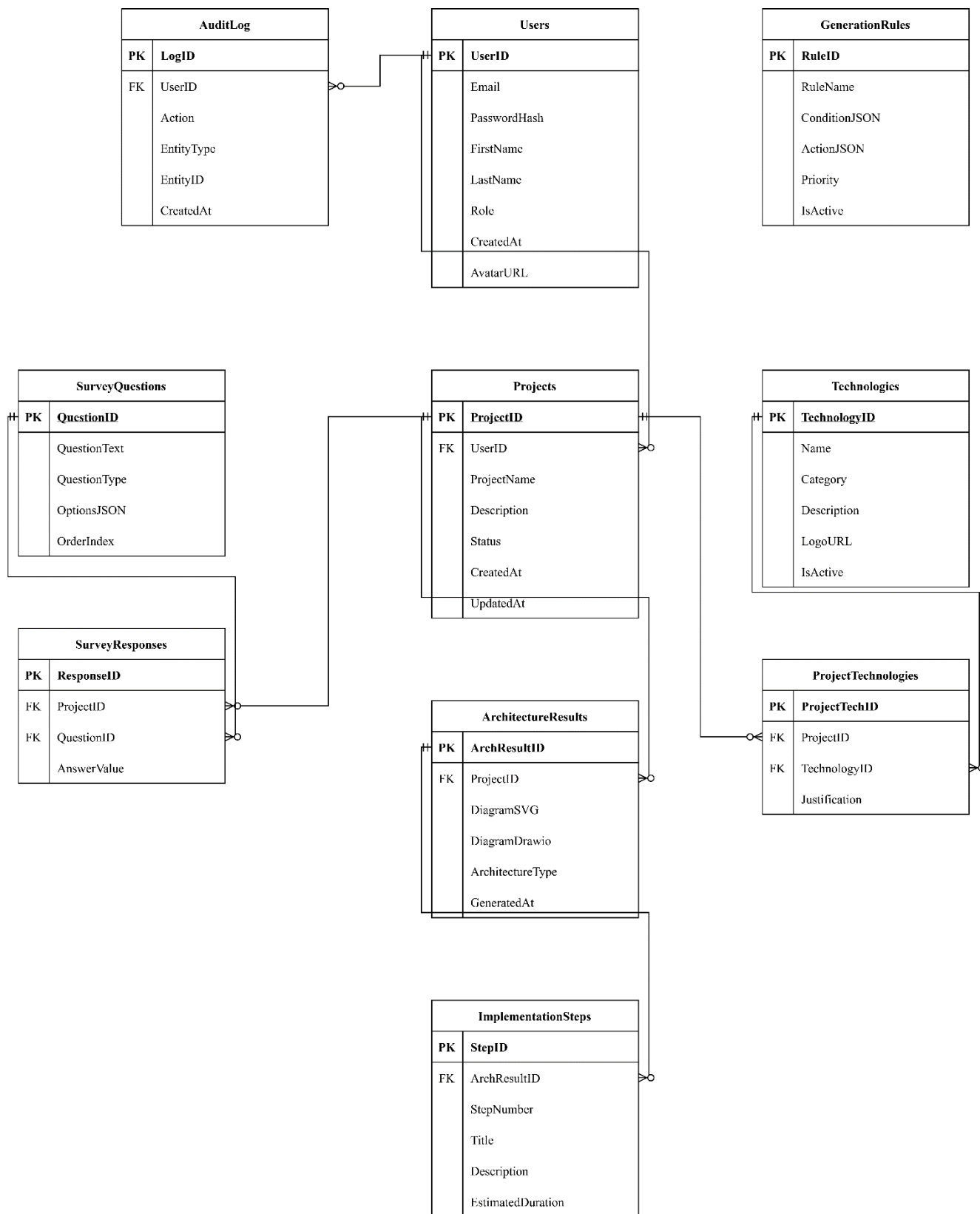
Список использованной литературы

- 1 React [Электронный ресурс]. – Режим доступа: <https://react.dev/> – Дата доступа: 11.05.2026.
- 2 PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/> – Дата доступа: 11.05.2026.
- 3 Nginx [Электронный ресурс]. – Режим доступа: <https://nginx.org/en/> – Дата доступа: 11.05.2026.
- 4 Docker Compose [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/compose/> – Дата доступа: 11.05.2026.
- 5 AWS Well-Architected Tool [Электронный ресурс]. – Режим доступа: <https://aws.amazon.com/ru/well-architected-tool/> – Дата доступа: 11.05.2026.
- 6 AWS [Электронный ресурс]. – Режим доступа: <https://aws.amazon.com/ru/> – Дата доступа: 11.05.2026.
- 7 Cloudcraft [Электронный ресурс]. – Режим доступа: <https://www.cloudcraft.co/> – Дата доступа: 11.05.2026.
- 8 Azure [Электронный ресурс]. – Режим доступа: <https://portal.azure.com/> – Дата доступа: 11.05.2026.
- 9 NodeJS [Электронный ресурс]. – Режим доступа: <https://nodejs.org/en/> – Дата доступа: 11.05.2026.
- 10 ExpressJS [Электронный ресурс]. – Режим доступа: <https://expressjs.com/> – Дата доступа: 11.05.2026.
- 11 REST API [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/node-js/rest-api-introduction/> – Дата доступа: 11.05.2026.
- 12 Vite [Электронный ресурс]. – Режим доступа: <https://vite.dev/> – Дата доступа: 11.05.2026.
- 13 Axios [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/html/what-is-axios/> – Дата доступа: 11.05.2026.
- 14 Drizzle [Электронный ресурс]. – Режим доступа: <https://orm.drizzle.team/> – Дата доступа: 11.05.2026.

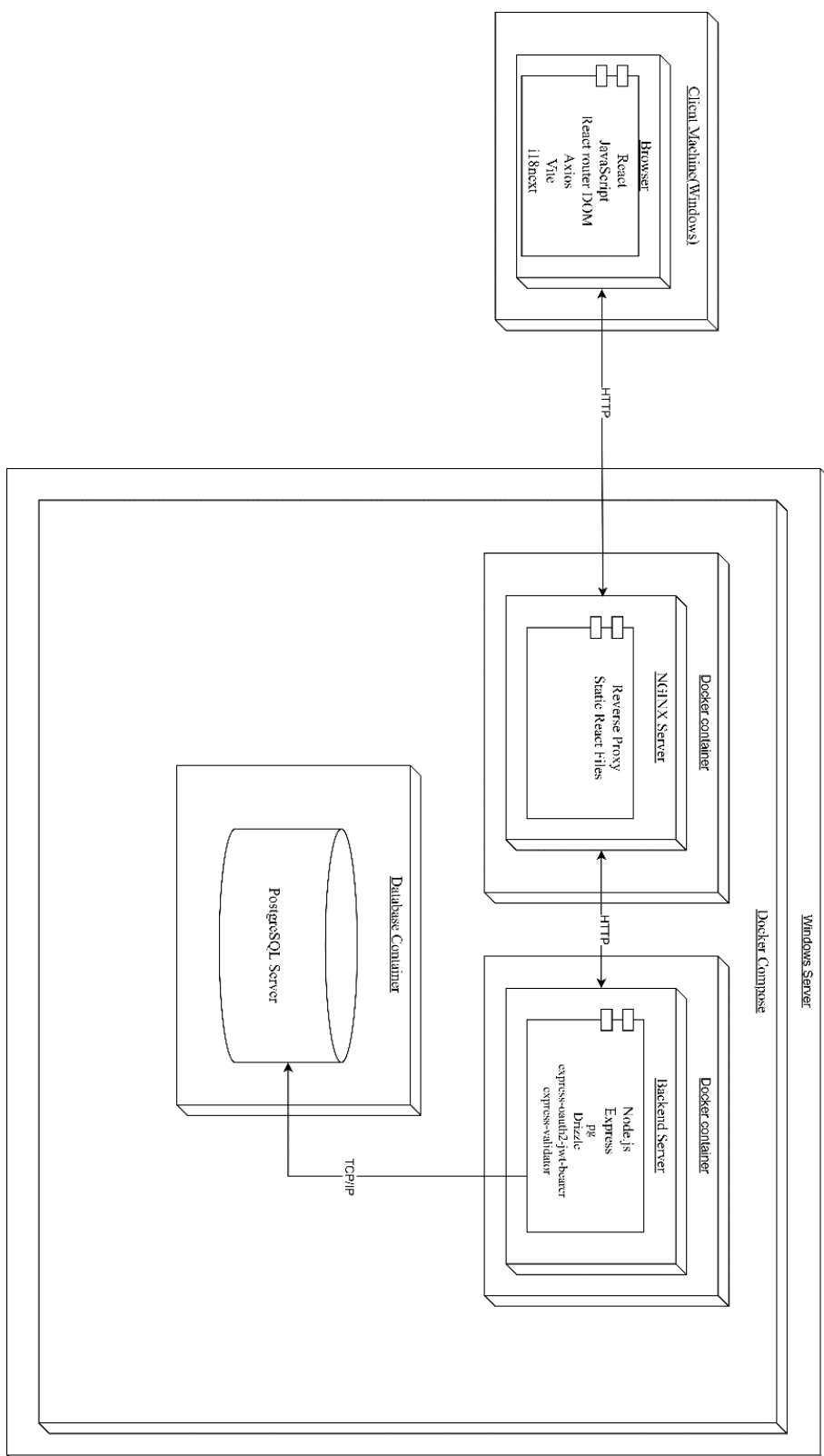
Приложение А



Приложение Б



Приложение В



Приложение Г

```
let pool = null;
let db = null;

function isDatabaseConfigured() {
  return Boolean(process.env.DATABASE_URL);
}

function getPool() {
  if (!isDatabaseConfigured()) {
    return null;
  }

  if (pool) {
    return pool;
  }

  const { Pool } = require("pg");

  pool = new Pool({
    connectionString: process.env.DATABASE_URL,
    ssl: process.env.DATABASE_SSL === "true" ? { rejectUnauthorized:
false } : false,
  });

  return pool;
}

function getDb() {
  if (!isDatabaseConfigured()) {
    return null;
  }

  if (db) {
    return db;
  }

  const { drizzle } = require("drizzle-orm/node-postgres");

  db = drizzle(getPool());

  return db;
}

module.exports = {
  getDb,
  getPool,
  isDatabaseConfigured,
};
```

Листинг Г.1 – Подключение приложения к базе данных

Приложение Д

```

const express = require("express");

const adminRouter = require("./routes/admin");
const authRouter = require("./routes/auth");
const healthRouter = require("./routes/health");
const questionnaireRouter = require("./routes/questionnaire");
const plansRouter = require("./routes/plans");
function createApp() {
  const app = express();
  app.use(express.json({ limit: "1mb" }));

  app.use((req, res, next) => {
    res.setHeader("Access-Control-Allow-Origin", "*");
    res.setHeader("Access-Control-Allow-Methods",
"GET, POST, PATCH, DELETE, OPTIONS");
    res.setHeader("Access-Control-Allow-Headers", "Authorization,
Content-Type");
    if (req.method === "OPTIONS") {
      return res.status(204).end();
    }
    return next();
  });
  app.use("/api/health", healthRouter);
  app.use("/api/auth", authRouter);
  app.use("/api/admin", adminRouter);
  app.use("/api/questionnaire", questionnaireRouter);
  app.use("/api/plans", plansRouter);
  app.use((req, res) => {
    res.status(404).json({
      error: "Not found",
      path: req.path,
    });
  });
  app.use((err, req, res, next) => {
    const statusCode = err.statusCode || 500;

    res.status(statusCode).json({
      error: err.message || "Unexpected server error",
      details: err.details || null,
    });
  });

  return app;
}

module.exports = {
  createApp,
};

```

Листинг Д.1 – Главный модуль серверного приложения

Приложение Е

```

const { auth } = require("express-oauth2-jwt-bearer");
const { createRepository } = require("../db/repositories/userRepository");
const userRepository = createRepository();
let checkJwt = null;
function isAuthenticated() {
    return Boolean(process.env.AUTH0_DOMAIN &&
process.env.AUTH0_AUDIENCE);
}
function getIssuerBaseURL() {
    if (!process.env.AUTH0_DOMAIN) {
        return null;
    }

    const normalizedDomain = process.env.AUTH0_DOMAIN.startsWith("http")
        ? process.env.AUTH0_DOMAIN
        : `https://${process.env.AUTH0_DOMAIN}`;

    return normalizedDomain.endsWith("/") ? normalizedDomain :
`${normalizedDomain}/`;
}

function getCheckJwt() {
    if (!isAuthenticated()) {
        return null;
    }

    if (!checkJwt) {
        checkJwt = auth({
            audience: process.env.AUTH0_AUDIENCE,
            issuerBaseURL: getIssuerBaseURL(),
        });
    }

    return checkJwt;
}

function requireAuth(req, res, next) {
    const activeCheckJwt = getCheckJwt();

    if (!activeCheckJwt) {
        return res.status(503).json({
            error: "Authentication is not configured on the server.",
            details: ["Set AUTH0_DOMAIN and AUTH0_AUDIENCE to enable protected
API routes."],
        });
    }

    return activeCheckJwt(req, res, next);
}

```



```

async function attachCurrentUser(req, res, next) {
  try {
    const claims = req.auth?.payload;

    if (!claims?.sub) {
      return res.status(401).json({
        error: "Authenticated token is missing the subject claim.",
      });
    }

    req.currentUser = await userRepository.upsertFromClaims(claims);
    return next();
  } catch (error) {
    return next(error);
  }
}

function requireAdmin(req, res, next) {
  if (!req.currentUser) {
    return res.status(401).json({
      error: "Current user context is required for admin access.",
    });
  }

  if (req.currentUser.role !== "admin") {
    return res.status(403).json({
      error: "Admin access is required for this route.",
    });
  }

  return next();
}

module.exports = {
  attachCurrentUser,
  isAuthenticated,
  requireAdmin,
  requireAuth,
};

```

Листинг Е.1 – Подключение стороннего сервиса для аутентификации

Приложение Ж

```

const { pgTable, bigserial, bigint, boolean, text, jsonb, integer,
timestamp, index, uniqueIndex } = require("drizzle-orm/pg-core");

const users = pgTable("users", {
  id: bigserial("id", { mode: "number" }).primaryKey(),
  auth0Sub: text("auth0_sub").notNull().unique(),
  email: text("email"),
  displayName: text("display_name"),
  role: text("role").notNull().default("user"),
  createdAt: timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
});

const generatedPlans = pgTable(
  "generated_plans",
  {
    id: bigserial("id", { mode: "number" }).primaryKey(),
    userId: bigint("user_id", { mode: "number" }).references(() =>
users.id, { onDelete: "cascade" }),
    planId: text("plan_id").notNull().unique(),
    projectName: text("project_name").notNull(),
    inputPayload: jsonb("input_payload").notNull(),
    summary: text("summary").notNull(),
    recommendationPayload: jsonb("recommendation_payload").notNull(),
    regionProfilePayload: jsonb("region_profile_payload"),
    roadmapPayload: jsonb("roadmap_payload").notNull(),
    developmentPlanPayload: jsonb("development_plan_payload"),
    costPayload: jsonb("cost_payload").notNull(),
    diagramPayload: jsonb("diagram_payload").notNull(),
    drawioXml: text("drawio_xml").notNull(),
    createdAt: timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  },
  (table) => ({
                                                                    createdAtIdx:
index("idx_generated_plans_created_at").on(table.createdAt),
                                                                    userCreatedAtIdx:
index("idx_generated_plans_user_created_at").on(table.userId,
table.createdAt),
  }),
);

const projects = pgTable(
  "projects",
  {
    id: bigserial("id", { mode: "number" }).primaryKey(),
    userId: bigint("user_id", { mode: "number" })
      .notNull()
      .references(() => users.id, { onDelete: "cascade" }),
    projectName: text("project_name").notNull(),
  },
);

```

```

        createdAt: timestamp("created_at", { withTimezone: true
    }).notNull().defaultNow(),
        updatedAt: timestamp("updated_at", { withTimezone: true
    }).notNull().defaultNow(),
    },
    (table) => ({
                                                                    userCreatedAtIdx:
index("idx_projects_user_created_at").on(table.userId,
table.createdAt),
                                                                    userProjectNameUniqueIdx:
uniqueIndex("uq_projects_user_name").on(table.userId,
table.projectName),
    }),
);

const planRuns = pgTable(
    "plan_runs",
    {
        id: bigserial("id", { mode: "number" }).primaryKey(),
        projectId: bigint("project_id", { mode: "number" })
            .notNull()
            .references(() => projects.id, { onDelete: "cascade" }),
        userId: bigint("user_id", { mode: "number" })
            .notNull()
            .references(() => users.id, { onDelete: "cascade" }),
        planId: text("plan_id").notNull().unique(),
        projectNameSnapshot: text("project_name_snapshot").notNull(),
        inputPayload: jsonb("input_payload").notNull(),
        summary: text("summary").notNull(),
        recommendationPayload: jsonb("recommendation_payload").notNull(),
        regionProfilePayload: jsonb("region_profile_payload"),
        roadmapPayload: jsonb("roadmap_payload").notNull(),
        developmentPlanPayload: jsonb("development_plan_payload"),
        costPayload: jsonb("cost_payload").notNull(),
        diagramPayload: jsonb("diagram_payload").notNull(),
        drawioXml: text("drawio_xml").notNull(),
        architectureStyle: text("architecture_style"),
        deploymentModel: text("deployment_model"),
        targetRegion: text("target_region"),
        monthlyEstimate: integer("monthly_estimate"),
        createdAt: timestamp("created_at", { withTimezone: true
    }).notNull().defaultNow(),
    },
    (table) => ({
                                                                    createdAtIdx:
index("idx_plan_runs_created_at").on(table.createdAt),
                                                                    userCreatedAtIdx:
index("idx_plan_runs_user_created_at").on(table.userId,
table.createdAt),
                                                                    projectCreatedAtIdx:
index("idx_plan_runs_project_created_at").on(table.projectId,
table.createdAt),

```

```

                                architectureStyleIdx:
index("idx_plan_runs_architecture_style").on(table.architectureStyle)
,
                                deploymentModelIdx:
index("idx_plan_runs_deployment_model").on(table.deploymentModel),
                                targetRegionIdx:
index("idx_plan_runs_target_region").on(table.targetRegion),
    }),
);

const planComponents = pgTable(
  "plan_components",
  {
    id: bigserial("id", { mode: "number" }).primaryKey(),
    planRunId: bigint("plan_run_id", { mode: "number" })
      .notNull()
      .references(() => planRuns.id, { onDelete: "cascade" }),
    componentCode: text("component_code").notNull(),
    createdAt: timestamp("created_at", { withTimezone: true
  }).notNull().defaultNow(),
  },
  (table) => ({
                                planRunIdx:
index("idx_plan_components_plan_run_id").on(table.planRunId),
                                componentCodeIdx:
index("idx_plan_components_component_code").on(table.componentCode),
                                runComponentUniqueIdx:
uniqueIndex("uq_plan_components_run_component").on(table.planRunId,
table.componentCode),
  }),
);

const technologyCategories = pgTable(
  "technology_categories",
  {
    id: bigserial("id", { mode: "number" }).primaryKey(),
    code: text("code").notNull().unique(),
    name: text("name").notNull().unique(),
    sortOrder: integer("sort_order").notNull().default(100),
    isActive: boolean("is_active").notNull().default(true),
    createdAt: timestamp("created_at", { withTimezone: true
  }).notNull().defaultNow(),
    updatedAt: timestamp("updated_at", { withTimezone: true
  }).notNull().defaultNow(),
  },
  (table) => ({
                                sortActiveIdx:
index("idx_technology_categories_sort_active").on(table.sortOrder,
table.isActive),
  }),
);

const technologies = pgTable(

```

```

"technologies",
{
  id: bigserial("id", { mode: "number" }).primaryKey(),
  name: text("name").notNull(),
  categoryId: bigint("category_id", { mode: "number" })
    .notNull()
    .references(() => technologyCategories.id, { onDelete: "restrict"
}),
  description: text("description"),
  logoUrl: text("logo_url"),
  isActive: boolean("is_active").notNull().default(true),
  createdAt: timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  updatedAt: timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
},
(table) => ({
  nameUniqueIdx: uniqueIndex("uq_technologies_name").on(table.name),
  categoryActiveIdx:
index("idx_technologies_category_active").on(table.categoryId,
table.isActive),
}),
);

const planRunTechnologies = pgTable(
  "plan_run_technologies",
  {
    id: bigserial("id", { mode: "number" }).primaryKey(),
    planRunId: bigint("plan_run_id", { mode: "number" })
      .notNull()
      .references(() => planRuns.id, { onDelete: "cascade" }),
    technologyId: bigint("technology_id", { mode: "number" })
      .notNull()
      .references(() => technologies.id, { onDelete: "restrict" }),
    justification: text("justification"),
    createdAt: timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  },
  (table) => ({
    planRunIdx:
index("idx_plan_run_technologies_plan_run_id").on(table.planRunId),
    technologyIdx:
index("idx_plan_run_technologies_technology_id").on(table.technologyI
d),
    uniquePairIdx:
uniqueIndex("uq_plan_run_technology").on(table.planRunId,
table.technologyId),
  }),
);

const regionDataCache = pgTable(
  "region_data_cache",

```

```

    id: bigserial("id", { mode: "number" }).primaryKey(),
    regionCode: text("region_code").notNull(),
    sourceName: text("source_name").notNull(),
    payload: jsonb("payload").notNull(),
    refreshedAt: timestamp("refreshed_at", { withTimezone: true
  }).notNull().defaultNow(),
    expiresAt: timestamp("expires_at", { withTimezone: true }),
  },
  (table) => ({
                                                                    regionSourceUniqueIdx:
uniqueIndex("uq_region_data_cache_region_source").on(table.regionCode
, table.sourceName),
                                                                    regionCodeIdx:
index("idx_region_data_cache_region_code").on(table.regionCode),
                                                                    expiresAtIdx:
index("idx_region_data_cache_expires_at").on(table.expiresAt),
  }),
);

const engineSettings = pgTable("engine_settings", {
  key: text("key").primaryKey(),
  valueJson: jsonb("value_json").notNull(),
  updatedBy: bigint("updated_by", { mode: "number" }).references(() =>
users.id, { onDelete: "set null" }),
  updatedAt: timestamp("updated_at", { withTimezone: true
}).notNull().defaultNow(),
});

const adminAuditLog = pgTable(
  "admin_audit_log",
  {
    id: bigserial("id", { mode: "number" }).primaryKey(),
    actorUserId: bigint("actor_user_id", { mode: "number"
}).references(() => users.id, { onDelete: "set null" }),
    action: text("action").notNull(),
    targetType: text("target_type").notNull(),
    targetId: text("target_id"),
    detailsJson: jsonb("details_json"),
    createdAt: timestamp("created_at", { withTimezone: true
}).notNull().defaultNow(),
  },
  (table) => ({
                                                                    createdAtIdx:
index("idx_admin_audit_log_created_at").on(table.createdAt),
  }),
);

module.exports = {
  adminAuditLog,
  engineSettings,
  generatedPlans,
  planComponents,
  planRunTechnologies,

```

```
planRuns,  
projects,  
regionDataCache,  
technologyCategories,  
technologies,  
users,  
};
```

Листинг Ж.1 – Создание базы данных